Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Hamza Alhalabi

29 April 2025

EdgeXcelerator: FPGA-Based CNN Hardware Accelerator for Image Classification in Edge Computing

Project supervisor: Professor Tomasz Kazmierski
Second examiner: Professor Mohammad Soorati

A project report submitted for the award of
BEng in Electrical and Electronic Engineering

Abstract

This project presents the design of an FPGA-based CNN inference accelerator optimised for high performance under low-resource constraints. The design approach adopts high-level synthesis through Intel's FPGA SDK for OpenCL to accelerate the CNN layers. A CNN based on the LeNet-5 architecture was trained using MATLAB, achieving an 82% reduction in network learnable parameters. The CNN classifies the 26 handwritten letters from the EMNIST letters dataset. Additionally, the CNN was quantised to INT8 using ONNX Runtime, resulting in a further 60.5% reduction in model size. Furthermore, all CNN layers were executed on the FPGA via OpenCL kernels incorporating multiple optimisation techniques. The final implementation achieved a throughput of 2.59 GOP/s, with an average frame rate of 12,135 images per second and 91.13% classification accuracy on a sample of 10,400 EMNIST letters test images. This represented a 20.3× speedup compared to the baseline performance on the ARM Cortex-A9 processor.

Statement of Originality

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have used Altera's design examples 'vector_add', 'TDFIR' [28], and 'autorun kernel' [29] to aid the OpenCL code development.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Acknowledgements

I would like to give special thanks to my supervisor Professor Tomasz Kazmierski for his insightful feedback, and patient guidance throughout this project, which helped shaping the technical direction and quality of this work. I would also like to thank my family and friends for being a constant source of encouragement during both the challenging and fulfilling moments of this work.

Contents

1 Introduction

Artificial Intelligence (AI) refers to machines' ability to perform tasks in an ‘intelligent’ manner. Machine Learning (ML) is a subset of AI, where a machine learns patterns from available data to make predictions about how to act, rather than being explicitly programmed to implement every type of task. Neural Networks are a subset of ML, and they excel in handling vast amounts of data such as images, video, and audio [1]. Convolutional Neural Networks (CNNs) are the type of neural network architecture that stands out for processing grid-like data, like images, through convolutions to perform tasks such as image classification.

While neural networks applications are gaining significance, they require substantial computational resources; therefore, advancing the hardware systems is essential to ensure fast and efficient execution. However, the limitations of Moore's Law and the end of Dennard scaling have made performance improvements through semiconductor scaling increasingly difficult. Additionally, due to the rising demand for AI services and the costs of AI data centres, federated computing on edge devices, like smartphones, became an attractive solution for cost-effectiveness, efficiency, and privacy, by eliminating the need to transmit user data to the cloud.

Hence, the response to these challenges has been the adoption of Application-Specific Integrated Circuits (ASICs), which are designed to maximise performance per watt and throughput while effectively managing computational demands. For instance, commercial companies are now advertising top-of-the-line products featuring *Neural Processing Units* (NPUs), which specialise in parallel processing of vast amounts of data more efficiently than CPUs or GPUs. Examples include Apple’s Neural Engine in the M-series chips [2], and Intel’s 200HX processors [3]. Nevertheless, the development of such custom architectures begins with prototyping on Field-Programmable Gate Arrays (FPGAs), due to their reconfigurability, reduced time-to-market, and the ability to validate hardware prior to costly ASIC fabrication.

With this context established, this project’s primary objective is to develop an **FPGA-based hardware accelerator** for CNN inference using the DE1-SoC development board. The project aims to accelerate a modified version of the LeNet-5 architecture, employing a hardware-software co-design approach to adapt to the DE1-SoC’s limited resources. Additionally, a central feature of this project is implementing the digital hardware design through *High-Level Synthesis* (HLS) using *OpenCL*, which is increasingly used in commercial projects, yet rarely used in undergraduate projects.

This project has three key **goals**. **First**, to modify and train a LeNet-5 CNN using MATLAB, with attention to constraints such as memory usage and the number of multiply-accumulate (MAC) operations to address the challenge of DE1-SoC limited hardware resources. **Second**, to implement the network using OpenCL by developing dedicated kernels for convolutional, pooling, and fully connected layers to execute on the Cyclone V FPGA. This also included building a host application on the ARM Cortex-A9 processor to manage kernel execution and data transfer. **Third**, to integrate these components to showcase inference results and benchmark its performance against a CPU-only implementation, with a focus on achieving high throughput.

The **stretch goals** were to enhance the CNN classification functionality and to implement hardware optimisations aimed at improving performance, provided that the initial prototype proved feasible. Given that the first integrated prototype could perform inference and fit within the available hardware resources, the CNN functionality was extended from a simple binary classification of ellipses to the 26 classes of the EMNIST letters dataset. In parallel, the hardware architecture was optimised, with the final design achieving a 2.47× higher throughput than a comparable a recent paper implementation [4].

The remainder of this report is structured as follows. **Chapter 2** presents the rationale for selecting FPGAs to accelerate CNNs, followed by a review of relevant literature on CNN acceleration methods. **Chapter 3** provides background on Intel's FPGA SDK for OpenCL. **Chapter 4** describes the process of modifying, training, and quantising the CNN models. **Chapter 5** details the final hardware architecture design and explains the optimisations adopted. **Chapter 6** details the testing methodology and evaluates the results. **Chapter 7** reflects on the use of HLS tools and project management. Finally, **Chapter 8** summarises the project's achievements and proposes future work directions.

2 Background and Literature Review

CNNs are structured networks, having different layers that perform dedicated tasks. The main layers include convolutional layers, pooling layers, and fully-connected layers, but different CNN architectures include more unique layers such as batch normalisation, residual connection layers, and more. This work assumes familiarity with CNNs; however, an overview is provided in Appendix A for reference if required.

2.1 Why FPGAs

CNNs are computationally intensive, requiring high throughput to meet application demands. Convolutional layers dominate computational time as seen in Fig. 2.1 [5], while fully-connected layers impose memory-fetching bottlenecks [1], [6]. For this reason, FPGAs are preferred due to their reconfigurability which enables them to address the processing challenges of each layer via tailored design. In contrast, classic CPU architectures, such as Von Neumann, execute programs sequentially. The processor fetches each instruction, decodes it, performs it, and iterates, where fetching the instructions limits the processing speed (memory-bound) [7]. In contrast, FPGAs use dataflow control to process data directly without relying on instruction memory, and their reconfigurability allows efficient operations such as adding two 8-bit numbers compared to the overhead of a CPU's fixed 64-bit architecture.

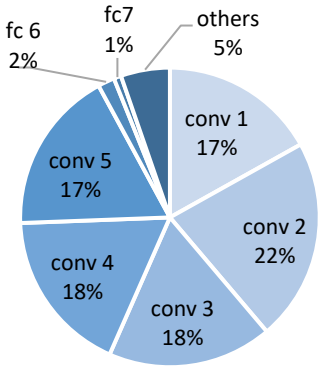


Figure 2.1: Distribution of computational time of each layer in an Imagenet CNN architecture on a GPU, reproduced from [5].

Alternatively, GPUs are commonly used for CNN acceleration due to their high throughput, such as the NVIDIA H100 Tensor Core GPU, which achieves up to 3,958 TeraFLOPs (floating-point operations per second) [8]. However, the high power consumption makes them unsuitable for power-constrained edge computing applications [9]. In contrast, FPGAs offer superior performance per watt and can be tailored to meet specific latency requirements, making them a more viable option. While both GPUs and FPGAs feature tensor cores (processing elements) to accelerate multiply-accumulate operations, FPGAs' flexible soft logic enables dataflow control which reduces latency, such as dynamically converting matrix inputs to vectors. Furthermore, FPGAs achieve

lower latency by positioning tensor blocks close to memory elements and directly connecting a tensor's output to the next tensor as illustrated in Fig. 2.2 [10].

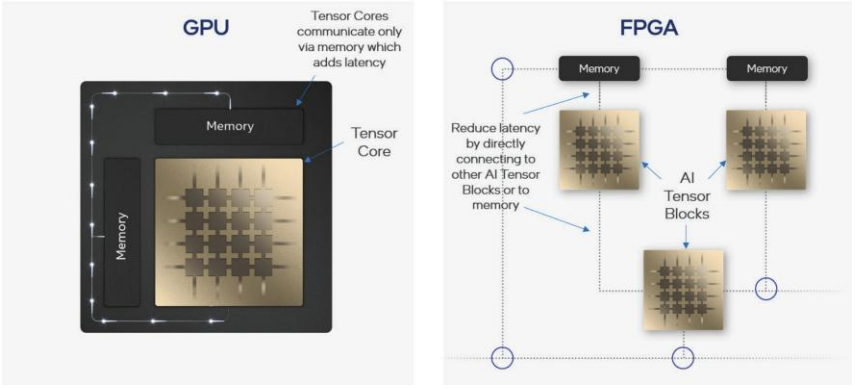


Figure 2.2: GPUs' tensor cores memory access compared to FPGAs' tensors lower latency memory due to direct connections, sourced from [10].

2.2 FPGA Acceleration Methods

Abdelouahab et al. [7] survey maps the main inference acceleration methods used as shown in Fig. 2.3, where the authors speculate that approximate computing methods will be key to accelerating CNNs in the upcoming years. This section explores the challenges and solutions associated with CNN acceleration.

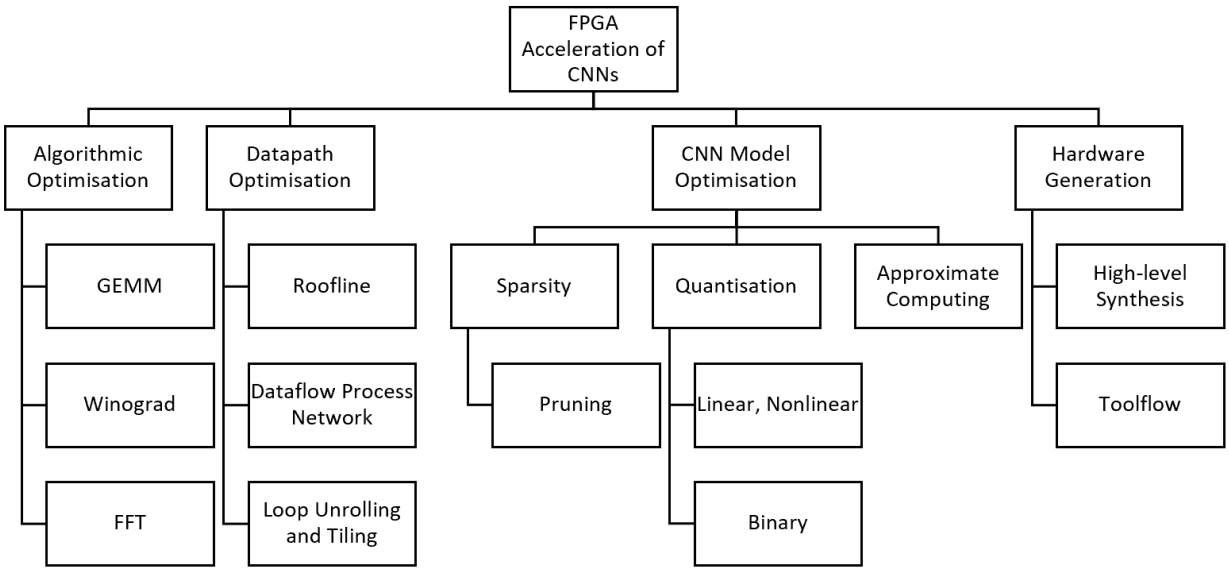


Figure 2.3: Map of CNNs inference acceleration methods, adapted from [7].

2.2.1 Algorithmic Optimisation Methods

Algorithmic optimisation methods target improving the efficiency of matrix multiplications by applying transforms to the data [1]. For instance, General matrix multiplications (GEMMs) are useful when processing a batch of feature maps to improve efficiency. In fully-connected layers, instead of loading the weights of each neuron for every input feature map, the feature maps can be batched into one matrix, allowing the neuron weights to be loaded once per batch.

On the other hand, Winograd transformation decreases the convolution complexity by reducing the multiplications to additions resulting in better performance. Yang et al.[9] employed Winograd transformation and resolved unaligned global memory access caused by the transformation by using alignment stream buffers, which maintained DRAM performance.

Alternatively, the Fast Fourier Transform (FFT) converts the time-domain image and filters to the frequency-domain, which converts the 2D convolutions to element-wise multiplication in the frequency-domain. This results in reduced time complexity from $O(U^2K^2)$ to $O(U^2 \log_2(U))$ [1], where U is the size of the output feature map and K is the filter size. Almorin et al. [11] implemented FFT transformations using High-level Synthesis tools and explored the advantages of parallelisation strategies to achieve high throughput.

2.2.2 Data Path Optimisation

Memory access is a significant bottleneck for CNN layers that need to fetch large numbers of weights and activations, making them memory-bound. A single multiply-accumulate (MAC) operation requires three memory reads and one write, as depicted in Fig. 2.4 [12]. For instance, fully-connected layers, with large weight matrices, are often memory-bound and can be optimised with batching to balance weight fetching and computation. Conversely, convolutions require significant computations, making them compute-bound [6].

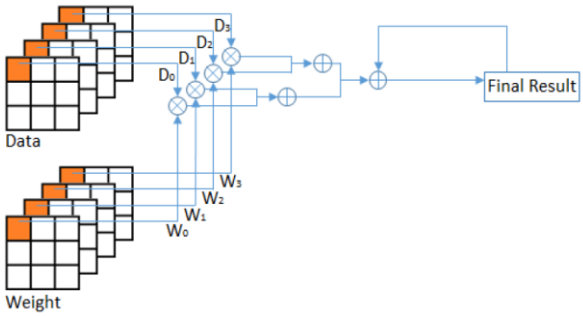


Figure 2.4: Multiply-Accumulate operations requiring three memory read for the weights, data (activations),and partial sum (Final Result buffer), and one write to the final result buffer, sourced from [12].

To fully use the logic and memory resources, Zhang et al. [13] proposed an analytical approach using the *roofline* model, shown in Fig. 2.5 [6]. Optimisation solutions such as *loop tiling* were employed to maximise the reusability of data, where the nested loops of a convolutional layer are divided into smaller tiles, this allows the weights and activations to fit on the on-chip buffers. Furthermore, for each CNN design solution, they performed quantitative analysis on the throughput and memory bandwidth then employed the roofline model to find the best-performing solution at the lowest resource cost.

The roofline model's x-axis represents operational intensity, an application property defined as the ratio of computations to off-chip memory communication (the parameters and activations fetched). The y-axis shows hardware performance, where low operational intensity indicates memory-bound constraints and high operational intensity reflects better data reuse, allowing full utilisation of hardware resources.

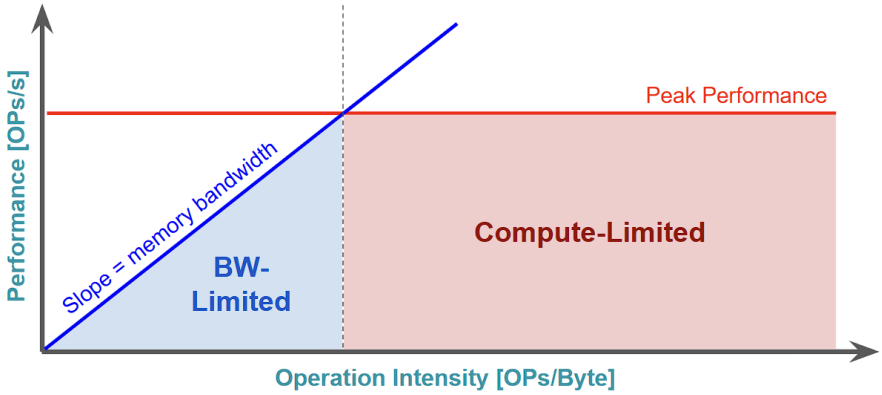


Figure 2.5: Roofline model showing the upper-performance limit of a system, constrained by computational and memory capabilities, sourced from [6].

Parallelism for CNN exists at various stages, for example, a batch of input images could be processed simultaneously, and the subsequent layers in the CNN could be pipelined by executing the next layer before the current layer finishes. On the convolutional layer level, four types of parallelism exist as listed in Table 2.1, which also allows for data reuse for improved operational intensity and throughput [7]. This parallelism could be implemented through *loop unrolling as* demonstrated in Listing 2.1. However, this comes at the cost of increased hardware utilization.

Table 2.1: Convolutional layer types of parallelism possibilities, summarised from [7].

Type of Parallelism	Description
Inter-feature-map	Each filter is multiplied with the same input image simultaneously.
Intra-feature-map	Multiple pixels within one output feature map are produced at the same time.
Inter-convolution	The convolution of each channel is performed independently.
Intra-convolution	Pipelining of 2D convolutions within each channel.

```
// Regular Loop
for (int i=0; i<0 ; i++){
    Sum += array[i];
}
```

```
// Unrolled Loop
Sum += array[0];
Sum += array[1];
Sum += array[2];
Sum += array[3];
```

Listing 2.1: Loop unrolling operation compared to regular loop iteration, where all elements are calculated simultaneously in the unrolled loop.

Nevertheless, accessing the operands from the DRAM is energy expensive; therefore, several memory hierarchy strategies reduce not only the time but also the energy associated with fetching the data. These include the use of local memory registers, global memory buffers, and the direct transfer of PEs' results to subsequent PEs. These methods could reduce the energy to fetch the data up to 2 orders of magnitude compared to fetching directly from DRAMs [1]. For instance, the activations of a fully-connected layer could be streamed simultaneously to all the processing elements while having the neurons' weights stored locally in register files, minimising the need for DRAM accesses.

Yang et al. [14] work focused on data path optimisation by implementing *image row broadcasting*, which optimised the convolutional dataflow on spatial architecture with 22x22 PEs, and increased data reuse while reducing data movement. Moreover, they

implemented *Zero Detection Technology*, which effectively skip the reading of unnecessary weights to reduce the number of off-chip DRAM accesses.

Yu and Li [15] employed a *ping-pong buffer* strategy to mitigate latency associated with data transfers. The ping-pong buffer effectively overlaps data loading with processing, ensuring that computation can proceed without waiting for memory transfers. However, they encountered limitations due to insufficient on-chip BRAM (Block RAM), which required multiple reloads of input feature maps and weights, adding significant overhead and impacting the efficiency gains from the parallelisation.

2.2.3 CNN Model Optimisation and Compression

Several techniques could be employed to reduce the size of a CNN, enabling its deployment in low-resource environments by addressing computational and memory constraints. For instance, floating-point representations of activations and weights provide high resolution but impose significant demands on computations and memory resources. To address this, *quantisation* converts operands to lower-precision formats with reduced bit-width, thereby relaxing resource requirements while aiming to minimise errors introduced by the quantisation [16].

Quantisation can map floating-point values to linear or nonlinear scales, resulting in fixed-point representations [1]. Alternatively, quantisation can map values to a binary scale (e.g., -x, +x), forming binary neural networks (BNNs). Post-quantisation, network's weights can be fine-tuned to preserve accuracy [7].

In contrast, pruning reduces the network size by eliminating less significant weights, and setting them to zero, which increases model sparsity (i.e., the proportion of zero multiplications) while having minimal impact on accuracy. Pruning methods are categorised into unstructured and structured approaches. Unstructured pruning achieves high compression rates, making it effective for fully-connected layers; however, it introduces irregular sparsity, leading to workload imbalances and reduced parallelism in convolutional layers. Conversely, structured pruning is hardware-friendly and optimised for convolutional layers, but it may remove critical weights, potentially diminishing accuracy. Song et al. [17] work achieved a balance between compression rate and hardware efficiency by applying regular pruning to convolutional layers for enhanced hardware performance while using unstructured pruning in fully-connected layers to maximise compression.

Another technique is *stochastic computing*, which represents numbers as bit-streams, where the proportion of 1's in the sequence encodes the number value. For instance, using a 10-bit stream, the number 0.6 can be represented as 1011011100. This representation enables operations like multiplication to be performed using simple logic gates, such as an AND gate, which effectively reduces logic and power utilisation [18].

2.2.4 High-Level Synthesis Tools

High-Level Synthesis (HLS) tools were introduced to simplify the process of designing hardware accelerators by providing a high level of abstraction for describing the hardware. The compilers aim to create efficient designs by optimising how operations are mapped and scheduled. Tools like Intel's OneAPI and Xilinx's Vivado enable faster development compared to writing RTL code manually. These tools use languages such as DPC++,

SYCL, and OpenCL to support heterogeneous applications across CPUs, GPUs, and FPGAs [19], [20]. However, these tools do not account for application structure, requiring designers' attention.

Since CNNs are structured and parameterizable with well-separated layers, toolflows have been developed to map CNNs directly from a high-level software definition onto customised FPGA hardware architectures. These toolflows enable developers to use FPGA accelerators without requiring expertise in hardware design. The toolflows, shown in Table 2.2, work in conjunction with the HLS tools and input interface frameworks. The interface frameworks, such as Caffe, are used to provide a high-level environment for defining, training, and exporting neural networks [21].

The generated hardware architecture can be classified into two main categories*: Streaming architectures* and *single computation engines*. Streaming architectures, such as fpgaConvNet, assign dedicated hardware blocks to each CNN layer, allowing pipelined execution across layers, which maximises parallelism but increases compilation time. In contrast, single computation engines, such as DeepBurning, share a flexible processing unit across layers, reducing resource demand through run-time reconfiguration of the FPGA, which reduces parallelism [21].

Table 2.2: List of Toolflows that map CNNs to FPGAs and their respective interface, reproduced from [21].

Toolflow Name	Interface	Year
fpgaConvNet	Caffe & Torch	May 2016
DeepBurning	Caffe	June 2016
Angel-Eye	Caffe	July 2016
ALAMO	Caffe	August 2016
Haddoc2	Caffe	September 2016
DnnWeaver	Caffe	October 2016
Caffeine	Caffe	November 2016
AutoCodeGen	Proprietary Input Format	December 2016
FINN	Theano	February 2017
FP-DNN	TensorFlow	May 2017
Snowflake	Torch	May 2017
SysArrayAccel	C Program	June 2017
FFTCodeGen	Proprietary Input Format	December 2017

Most relevant to this project, Ngo et al. [22] work employed Intel’s FPGA SDK for OpenCL to accelerate object detection by implementing a modified Tiny-YOLO-v2 CNN architecture on the resource-constrained DE1-SoC development board. By adopting half-precision floating-point data types, they achieved 90% accuracy, enhancing detection performance within the board's limitations.

3 Intel FPGA SDK for OpenCL

The Intel FPGA SDK for OpenCL programming model has three main components. First, the **host application**, written in C++, runs on the ARM-based Hard Processor System (HPS) of the DE1-SoC. This application, executed on Linux, is responsible for managing the accelerator, where it allocates global memory buffers, handles data transfers between host and FPGA, and launches OpenCL kernels via command queues. Second, the **OpenCL kernel(s)** define the behaviour of the accelerator and are synthesised into custom logic on the FPGA fabric, as illustrated in Fig. 3.1 [23]. Finally, the **Board Support Package** (BSP) provides the SDK compiler with hardware-specific configurations for the target platform.

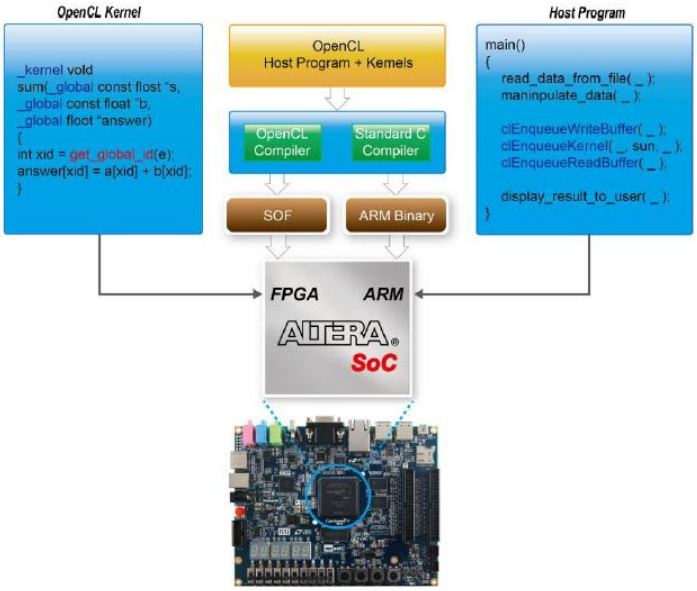


Figure 3.1: OpenCL workflow illustrated on the DE1-SoC, sourced from [23].

OpenCL expresses parallelism through the concepts of global and local index spaces, which define how kernels are executed across parallel threads, known as *work-items*. The *global size* refers to the total number of *work-items* for a given problem and can be defined in one, two, or three dimensions. For example, processing a 1920×1080 image corresponds to a global size of 2,073,600 work-items in a 2D space. These work-items are grouped into *local work-groups*, which are smaller batches that execute together. Each *work-group* is scheduled to run on a *compute unit*, which is a physical instance of the kernel on the FPGA. Importantly, work-items within a work-group can be **synchronised** using barrier instructions, while synchronisation across different work-groups is not guaranteed.

OpenCL kernels typically use the “get_global_id(dim)” function to identify the current work-item’s index in each dimension. This index determines the specific data element a work-item operates on, enabling a data-parallel model. For example, in a simple element-wise multiplication of two arrays, each work-item retrieves its unique index using “get_global_id(0)” and multiplies the corresponding elements as shown in Listing 3.1 [24]. The level of parallel execution achievable on the FPGA depends on the available hardware resources, as specified by the BSP, and on how the compiler pipelines and replicates kernel logic.

```
// kernel for adding two input vectors
__kernel void vector_add( __global const float *x, __global const float *y, __global float *restrict z)
{   // get index of the work item
    int index = get_global_id(0);
    // add the vector elements
    z[index] = x[index] + y[index];
}
```

Listing 3.1: "vector_add" example of a simple OpenCL kernel that performs multiplication of two vectors in a 1D index space. The __global qualifiers indicate that the arguments point to global memory objects, sourced from [24].

On the host side, key OpenCL concepts are involved. The *device* refers to the FPGA, while the host refers to the ARM processor. A *context* defines the set of devices and associated memory objects that can be shared across them. A *command queue* is created to submit operations such as memory transfers and kernel executions to the FPGA. The host application compiles the OpenCL program, which includes loading the ".aocx" bitstream to reconfigure the FPGA. It then creates memory buffers, enqueues data transfers to and from the FPGA, sets kernel arguments, launches the kernel by enqueuing it, and waits for all operations to complete using synchronisation primitives.

4 CNN Architecture Design

The LeNet-5 architecture [25], typically used to classify greyscale handwritten digits, was adopted primarily due to the FPGA's limited resources, as it features simple architecture, requiring less memory and fewer computations compared to alternative architectures. However, the original LeNet-5 architecture has 60,856 learnable parameters (weights and biases) and requires 415,680 MAC operations. This risked exceeding the DE1-SoC FPGA's BRAM capacity (16KB) and the available hardware resources, thus making it infeasible to run the complete network entirely within the FPGA without resorting to off-loading data to global memory. To address this, the design space was explored during training by tuning the hyperparameters, including convolution filter size, pooling window size, stride, and the number of neurons in the fully-connected layers. Further reduction in the network size was achieved by quantising the CNN parameters to integer values.

4.1 Trained CNNs

The CNN design workflow employed MATLAB's Deep Learning toolbox to train the CNNs and experiment with architectural changes by parameterising the CNN. The toolbox training options were explored, where solver options like **SGDM** (Gradient Descent with Momentum) and **Adam** (adaptive moment estimation) were tested with a range of initial learning rates. Experiments results led to using the "Adam" solver, with 0.001 initial rate as it offered faster convergence to higher validation accuracy. Moreover, training was configured over 10 epochs, shuffling the data every epoch, evaluating validation performance every 40 iterations, and the best validation accuracy CNN was retained.

The first modified version of LeNet-5 was trained to perform binary classification of hand-drawn ellipses as a starting point. This helped demonstrating the feasibility of implementing the accelerator using OpenCL and verified that all CNN layers could be mapped to the FPGA. The ellipses CNN architecture, detailed in Table 4.1, was optimised to balance accuracy and network size while reducing the number of required MAC operations. To train the ellipses CNN, the hand-drawn shapes dataset from [28] was selected due to its greyscale format, which aligns with the original LeNet-5 architecture.

Table 4.1: Optimised architectures of the Ellipses CNN and the EMNIST CNN, detailing their layer configurations, and classification accuracy on the test dataset. The total number of learnable parameters and MAC operations are included as indicators for hardware resource demands.

Layer	Ellipses CNN	EMNIST CNN
Input layer	28×28 image	28×28 image
Convolution 1 (ReLU)	3×3 kernel, 5 channels, stride 1	3×3 kernel, 5 channels, stride 1
Maxpool 1	2x2 pooling, stride 2 , no padding	2x2 pooling, stride 2 , no padding
Convolution 2 (ReLU)	8×8 kernel, 32 channels, stride 1	8×8 kernel, 16 channels, stride 1
Maxpool 2	2x2 pooling, stride 2 , no padding	2x2 pooling, stride 2 , no padding
Fully-Connected layer 1	5 neurons	30 neurons
Fully-Connected layer 2	2 neurons	26 neurons
Prediction Layer	SoftMax	SoftMax
Total Learnables	10,979	10,582
Total MACs	400,510	200,880
Accuracy (Test dataset)	92.68%	90.27%

Following the proof-of-concept implementation of the accelerator using the Ellipses CNN, a more advanced CNN was developed based on the same architecture to classify the EMNIST letters dataset. This dataset comprises 145,600 greyscale images of uppercase and lowercase handwritten letters across 26 classes, divided into 99,840 training samples, 24,960 validation samples, and 20,800 test samples [29]. The upgraded EMNIST CNN was used in the final version of the accelerator to demonstrate its capability to handle a more complex classification task while remaining within the same hardware constraints. The EMNIST CNN was further optimised to reduce the number of MAC operations by half compared to the Ellipses CNN, while maintaining high classification accuracy. It achieved 90.27% test accuracy, as shown in the confusion matrix in Fig. 4.1, with most misclassifications occurring between visually similar characters such as ‘I’ and ‘l’.

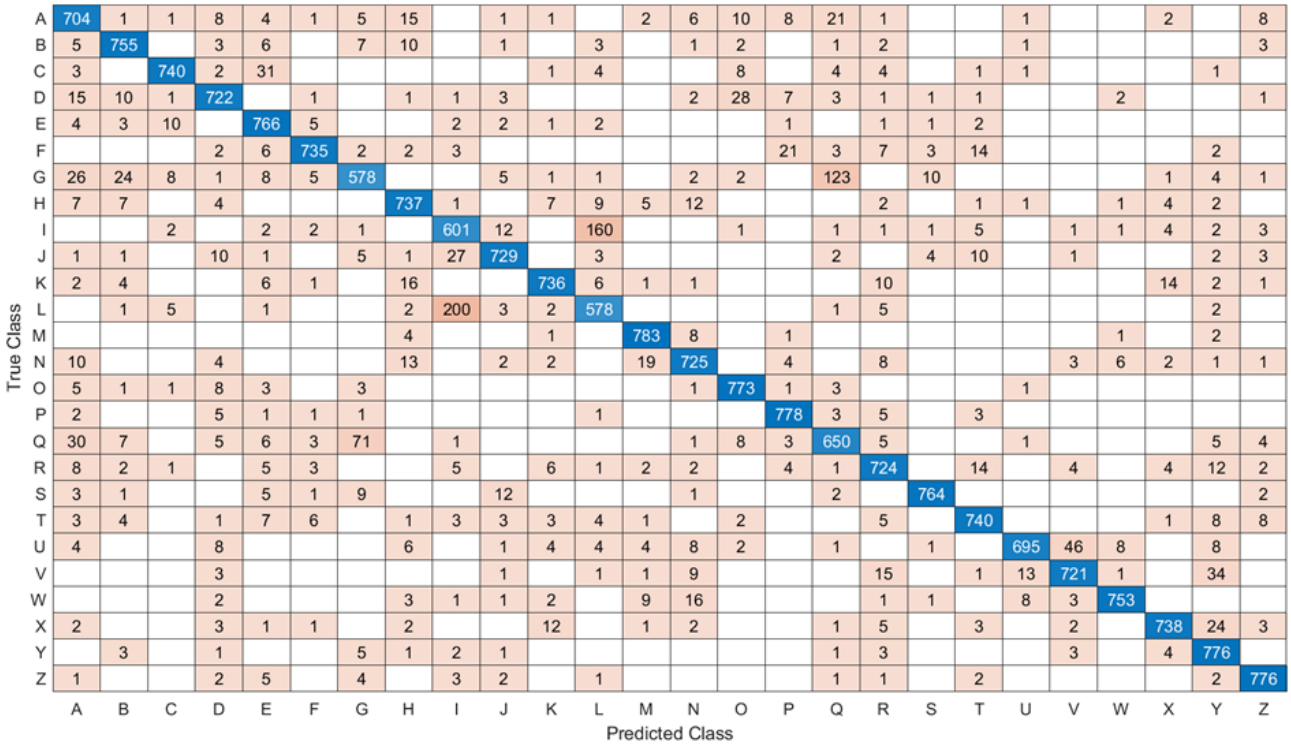


Figure 4.1: Confusion matrix of the EMNIST CNN, showing classification across 26 letter classes. Correct predictions appear along the diagonal, while off-diagonal entries indicate misclassifications.

4.2 CNN Quantisation

To enable deployment of the CNN on the FPGA with reduced hardware requirements, quantisation of the network parameters was required. Initially, MATLAB’s Deep Network Quantizer tool was explored; however, it lacked the capability to export the quantised parameters for external hardware implementation. Therefore, the Open Neural Network Exchange (ONNX) [26] was adopted to convert the trained networks into a portable format. Additionally, ONNX Runtime [27] was used to perform *post-training quantisation* and export the quantised parameters.

Quantisation was applied to the EMNIST CNN, where weights were quantised to signed 8-bit integers (INT8), while biases remained in 32-bit integer format (INT32). Biases were kept at 32-bit because the multiplication of two INT8 values (e.g. 127 × 127 = 16,129) yields an INT16 result, and accumulating two INT16 results requires INT32 accumulator to avoid overflow. Hence, the minimum width requirements of the accumulator memory align with the biases' bit-width, which also helps preserving the biases accuracy.

Before applying quantisation, design options had to be selected to ensure efficient hardware deployment while maintaining classification accuracy. These include the quantisation type, scaling method, granularity, and calibration technique. Table 4.2 summarises the configurations adopted in this work. A *per-layer static quantisation* approach was employed because each CNN layer operates over a distinct value range, making per-layer quantisation effective in preserving numerical accuracy. Although per-layer quantisation introduces overhead through intermediate re-quantisation nodes, it still provides faster inference than dynamic quantisation, which computes scales and zero-points at runtime. Moreover, this approach fits the layer-wise kernel architecture of the accelerator, simplifying the integration of quantisation nodes.

Table 4.2: Summary of selected quantisation design choices and their corresponding ONNX Runtime configurations.

Design Option	Choice	ONNX RT Configuration
Quantisation Type	Static Quantisation	quantize_static()
Range Type	Asymmetric linear	QuantizationMode.QLinearOps
Granularity	Per-Layer	– (default)
Calibration Method	Cross-Entropy	CalibrationMethod.Entropy

Additionally, *asymmetric linear quantisation* was used, as it better accommodates the varying numerical value distributions across layers while reducing the likelihood of clipping at the quantisation range extremes (127, −128). Subsequently, determining the optimal quantisation ranges, i.e. computing the *scales* and *zero-points*, could be implemented using methods such as *Min-max*, *Percentile*, or *Cross-Entropy*. These methods implicate trade-offs between clipping and rounding errors. The *Cross-Entropy* method was chosen, as it prioritises minimising quantisation error in the most critical network values, such as neuron activations in the output layer, thereby preserving classification accuracy [26]. Furthermore, the calibration data were taken from the EMNIST letters training dataset.

The quantisation of a layer's input feature map from 32-bit floating-point to 8-bit integer is defined as equation (1). Similarly, the quantisation of weights is defined in equation (2), where S is the corresponding scale (a floating-point value) and Z is the associated zero-point (an 8-bit integer).

$$I_{8bit} = \frac{I_{32bit}}{S_I} + Z_I \tag{1}$$

$$W_{8bit} = \frac{W_{32bit}}{S_W} + Z_W \tag{2}$$

When a MAC operation is computed for a convolutional or fully connected layer, the accumulated 32-bit partial sum is calculated using equation (3). Following accumulation, the partial sum must be re-quantised to match the scale of the subsequent layer using equation (4), where S_{Next} and Z_{Next} denote the scale and zero-point of the next layer, respectively.

$$P_{sum\_32bit} = \sum I_{32bit} \times W_{32bit} = (S_I \times S_W) \sum\{(I_{8bit} - Z_I) \times (W_{8bit} - Z_W)\} \tag{3}$$

$$P_{sum\_8bit} = \frac{P_{sum\_32bit}}{S_{Next}} + Z_{Next} \tag{4}$$

By substituting equation (3) into equation (4), the scaling factors can be combined into a single precomputed rescaling factor "M" as in equation (5), and the final re-quantisation formula becomes as in equation (6). This re-quantisation procedure described in equations (1)–(6) was applied consistently across all convolutional and fully-connected layers.

Furthermore, following the MAC operations and bias addition, the result was clamped to the 8-bit range (−128, 127) to prevent overflow and ensure valid output.

$$M = \frac{S_I \times S_W}{S_{Next}} \tag{5}$$

$$P_{sum\_8bit} = M \times \sum\{(I_{8bit} - Z_I) \times (W_{8bit} - Z_W)\} + Z_{Next} \tag{6}$$

To deploy the quantised EMNIST CNN, the weights, biases, scales, and zero-points for each layer were exported to header files. Additionally, to assess the quantisation impact, a quantitative comparison between the floating-point and quantised EMNIST CNN models, presented in Table 4.3, shows that quantisation reduced the network size by approximately 60.5% with only a minor accuracy decrease of 0.09%, which validates the effectiveness of the chosen design approach.

Table 4.3: Comparison between the floating-point and quantised EMNIST CNN models, showing their size and accuracy evaluated on the EMNIST letters test dataset.

Measure	Floating-Point CNN	Quantised CNN
Model Size	42.58 KB	16.80 KB
Accuracy (EMNIST Letters Dataset)	90.27%	90.18%

5 Hardware Architecture Design

This chapter presents the final hardware architecture developed to accelerate the quantised EMNIST CNN inference using Intel's FPGA SDK for OpenCL. To support the development and evaluation process, the experimental setup summarised in Table 5.1 was chosen given the available support and documentation. An overview of how to use this workflow is provided in Appendix E.

Table 5.1: List of experimental setup tools and their purposes.

Tools	Purpose
DE1-SoC Development Board	Runs the host and kernel code design.
MicroSD Card + Win32 Disk Imager	To provide the Linux image.
Intel FPGA SDK for OpenCL 18.1 Intel Quartus Prime Std Edition 18. I Intel SOC EDS 18.1 DE1-SoC Board Support Package Microsoft Visual Studio Code	Programming and compilation environment.
PuTTY + USB-UART Interface	To write to the Linux command-line.
Ethernet cable + WinSCP	To copy compiled files to the DE1-SoC.

Initially, the development began by experimenting with Altera's 'vector_add' example to perform a single neuron operation. This served as a foundation for understanding OpenCL and was then scaled into a fully connected layer kernel. The reuse of the example code here was limited to initialisation and buffer management components. Moreover, convolution kernels were next implemented and integrated into a host program that executed the remaining layers, which enabled early demonstration of inference capabilities. Subsequently, development advanced by gradually offloading the remaining CNN layers from the host onto the FPGA, initially employing floating-point accelerators before arriving at the final quantised accelerator implementation. The following sections present the final hardware architecture and the optimisation techniques that led to it.

5.1 The Final Hardware Architecture

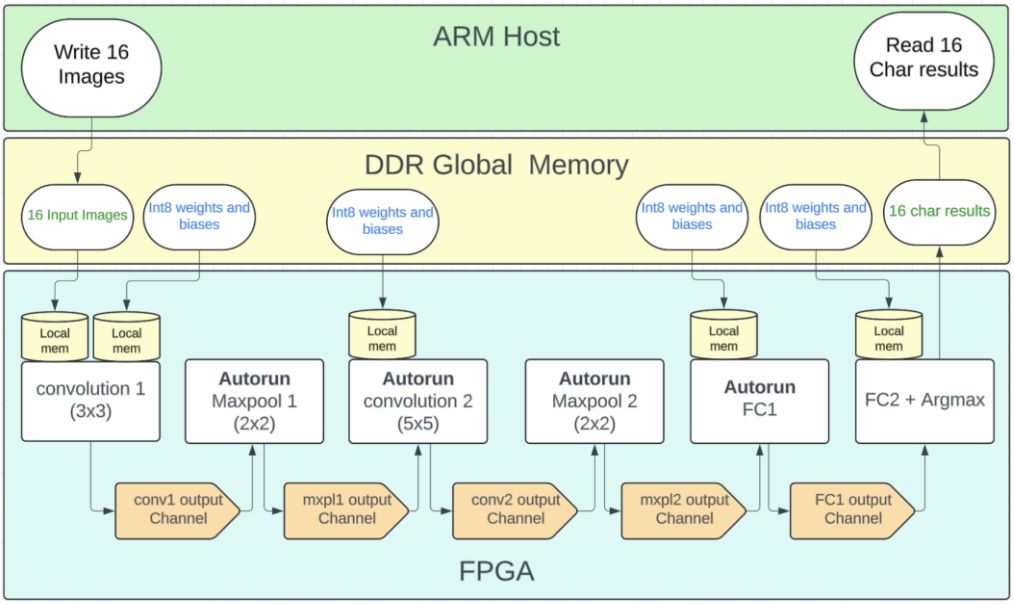


Figure 5.1: The final hardware design top-level view. The white boxes in the FPGA area represent the kernel instances, and 'local mem' units represent registers used to cache CNN parameters from constant global memory.

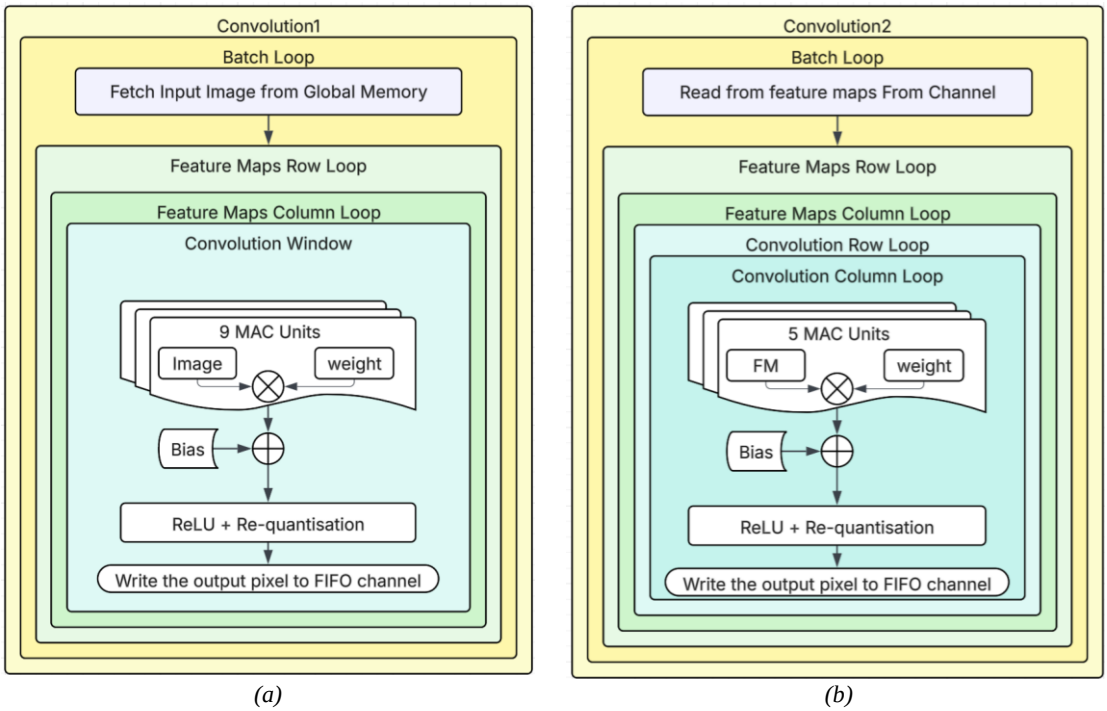
5.1.1 Control flow and Global Memory Management

The highest-performing accelerator adopts a streaming architecture, implemented as a pipelined single work-item kernel, as depicted in Fig. 5.1. The heterogeneous computation works as follows. The host program manages the accelerator by acting as the control interface and data source for input. First, a batch of 16 readily-quantised images on the host is written to the FPGA's global memory. Then, the host enqueues the first kernel, 'Convolution1', as a single work-item using the command queue. After 'Convolution1' processes the batch, control is handed over to a hardware scheduler on the FPGA, which automatically runs the intermediate kernels in turn, as they are given *autorun* attribute.

These kernels collectively implement convolutional, pooling, and fully connected layers, up to the final classification stage. Afterwards, the host enqueues the final kernel, 'FC2+Argmax', which is not autorun, as it must return predictions to the host via global memory. Once it completes, the host reads the output buffer containing the predicted class labels (each an 8-bit character), corresponding to the input image batch.

5.1.2 Convolution Kernels

In 'Convolution1' kernel, for each item in the batch, a single image is first prefetched into a BRAM unit. It then performs a 3×3 convolution with each of the five weight matrices to generate the corresponding feature maps. To improve throughput, the two innermost loops spanning the convolution window are fully unrolled using '*#pragma unroll*', which allows all nine pixel-weight multiplications to execute in parallel. This parallelism is made possible by preloading the required pixels into distinct BRAM locations, while the weights and biases are prefetched from constant global memory into local registers. Next, the resulting partial sums are accumulated in 32-bit registers and combined with the respective bias terms, as illustrated in Fig. 5.2(a). It should be noted that feature maps throughout the accelerator are processed in column-major order following MATLAB's method for storing data.



(a) *(b)*

Figure 5.2: Block diagrams of 'Convolution1' kernel (a) and 'Convolution2' kernel (b). The diagrams illustrate the loop hierarchies, data movement, and parallelism achieved through simultaneous operation of multiple MAC units.

Subsequently, re-quantisation and ReLU activation are applied without using floating-point operations. Specifically, after adding the bias, the rescaling factor M from equations (5) and (6) is approximated via an integer multiply-and-shift operation. For instance, in Listing 5.1, the constant 374 and a right shift by 17 together approximate the precomputed rescaling factor 'M=0.0028452151' in in 'Convolution1' kernel. Hence, multiplying the 32-bit partial sum by 374 and then dividing by 2^{17} is equivalent to applying the original floating-point rescaling. This transformation, which is applied similarly in both convolutional and fully connected layers, enables purely fixed-point arithmetic and thus avoids the generation of floating-point hardware. Finally, 'Convolution1' output feature maps are written to an 8-bit FIFO channel and consumed by the Maxpool1 kernel in a pipelined fashion.

kernels.cl

```
// Floating-point operation (old implementation)
int conv1_out = (int)(conv1_psum * M) + relu1_zero_point;

// Fixed-point Implmentation (Used in the final design)
int conv1_out = ((conv1_psum * 374) >> 17) + relu1_zero_point;
```

Listing 5.1: Comparison between the original floating-point implementation and the final fixed-point implementation of the re-quantisation step in Convolution1. The fixed-point version replaces the floating-point rescaling factor M with an integer multiply-and-shift operation.

Architecturally, "Convolution2" mirrors "Convolution1", as illustrated in Fig. 5.2(b), but operates as an autorun kernel and is adapted to meet higher computational demands. It processes five input channels and computes sixteen output feature maps. Since each output pixel results from MAC operations across all five input channels, fully unrolling the convolution window becomes more taxing. This is because unrolling increases the number of distinct BRAM units required to enable parallel access. Hence, to manage limited hardware resources, only the inner (column-wise) loop is fully unrolled, while the outer loops are pipelined. This selective unrolling balances parallelism with resource constraints while maintaining performance.

5.1.3 Maxpooling kernels

The Maxpooling kernels ("Maxpool1" and "Maxpool2") are both pipelined autorun kernels. Each kernel begins by reading a batch of feature maps from the preceding convolution layer via an 8-bit FIFO channel. These feature maps are cached in BRAM for parallel access. For each image in the batch and for each feature map, a 2×2 max-pooling operation is applied to down-sample the spatial resolution. In particular, the pooling window is processed using fully unrolled nested loops, allowing all four pixel comparisons to execute in parallel. Afterwards, the maximum value within each window is selected and written to a downstream 8-bit wide FIFO channel, as illustrated in Fig. 5.3. Moreover, since Maxpooling only selects the maximum value among input pixels and does not alter the scale of the data, no re-quantisation is applied.

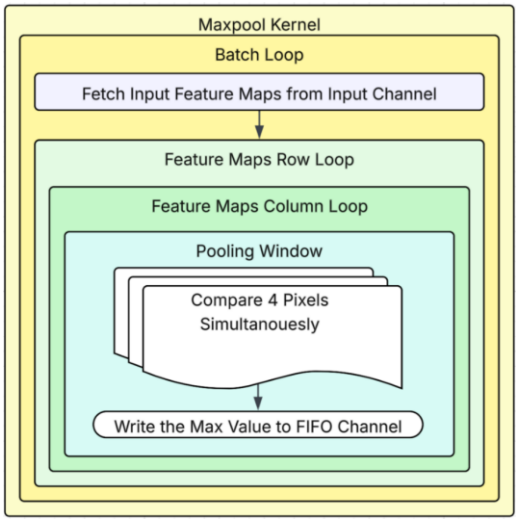


Figure 5.3: Block diagram showing the loop hierarchies, parallelism, and data movement in the Maxpooling kernels.

5.1.4 Fully-Connected and Prediction layers.

Following ‘Maxpool2’, the output feature maps are streamed directly into the ‘FC1’ kernel through a FIFO channel, eliminating the need for explicit vectorisation of the data. Upon receiving a complete input vector of size 256 per image, ‘FC1’ stores it in a BRAM buffer. For each image in the batch, it computes activations for the 30 output neurons. To balance resource constraints with performance, the inner MAC loop is unrolled by a factor of 3, enabling moderate parallelism within hardware limits. After accumulating the weighted sum, a bias is added, and the result is re-quantised using fixed-point arithmetic before being written to the output FIFO channel.

This output is finally consumed by ‘FC2 + Argmax’ kernel, which incorporates the argmax operation. For each image in the batch, the kernel reads the 30 input activations from ‘FC1’ into local BRAM. It then iterates through the 26 output neurons, performing the MAC loop using the stored activations. Like ‘FC1’, the MAC loop in ‘FC2’ is unrolled with a factor of 3.

The argmax functionality is embedded directly within the neurons loop of ‘FC2’ by tracking the highest accumulated sum ‘max_val’ and corresponding neuron index ‘pred_idx’ as shown in Listing 5.2. This serves as an efficient, hardware-integrated implementation of an argmax layer, embedded directly within the neurons’ loop. Upon completion, the final predicted index, offset by +1 to match label encoding, is stored as an 8-bit character in global memory.

```
// Inside batch loop of FC2+Argmax kernel
for (int neuron = 0; neuron < FC2_OUT_SZ; neuron++){
  int psum = 0;
  #pragma unroll 3
  for (int i = 0; i < FC2_IN_SZ; i++){ /*MAC operation*/  }
  psum += fcn2_biases[neuron]; // Bias addition
  if (psum > max_val){ //Argmax functionality
    max_val = psum;
    pred_idx = neuron;}
}
// Added +1 because indexing starts at zero, and letters labels start from 1 to 26.
prediction_buf[b] = (unsigned char)(1 + pred_idx); //prediction_buf points to global memory.
```

Listing 5.2: Snippet of ‘FC2+Argmax’ kernel showing embedded Argmax operation within the neurons’ loop.

5.2 Optimisations Rational

The first major optimisation following the proof-of-concept prototype was the quantisation of the EMNIST CNN hardware implementation. While this modification did not directly reduce inference time significantly, it lowered the hardware resource utilisation. Specifically, replacing floating-point arithmetic with fixed-point operations eliminated the need for floating-point units in MAC operations, thereby allowing resources to be reallocated for more loop unrolling. The impact of this substitution was assessed using the compiler-generated HTML report, with Table 5.2 summarising the estimated resource usage of 32-bit integer versus 32-bit floating-point operations.

Table 5.2: Comparison of hardware resource usage between 32-bit integer and 32-bit floating-point operations.

Operation	ALUTs	FFs	DSPs
32-bit Integer Add	17	0	0
32-FP Add	995	717	0
32-bit Integer Multiply	41	0	1
FP Multiply	274	240	1
Integer Compare	35	1	0
FP Compare	105	8	0

In addition to improving logic utilisation, quantisation reduced both global and local memory traffic. Since all transferred feature maps in the quantised design are stored as 8-bit values, each image required moving only 51.5 Kbit of feature map data. In contrast, the floating-point design required approximately 206 Kbit. This 4-fold reduction per image scales linearly with batch size; hence, a batch of 16 images benefits from a 64-fold decrease in memory transfer volume. Furthermore, all input images are pre-quantised on the host side before being transferred to the FPGA, thereby minimising data movement over the relatively slower global memory interface.

Moreover, the choice of 17-bit shift for the re-quantisation factor fixed-point representation was specifically chosen based on the magnitude of the rescaling factors used in the network (e.g., M=0.0028452151, M2 =0.0015106803). These small values required sufficient fixed-point precision to minimise quantisation error. Hence, using 17 fractional bits yields a resolution of approximately 7.63×10^{-6}, which proved adequate for accurately representing the scaling factors. Furthermore, increasing the shift width beyond this would have unnecessarily increased hardware resource usage without meaningful gains in accuracy.

Early hardware implementations scheduled kernels via the host using a command queue. Initially, kernels were configured as single work-groups with a three-dimensional ‘NDRange’, where each work-item processed a spatial computation (e.g., a convolution output pixel), identified by its (feature map, row, column) coordinates using ‘get_global_id()’. Although this configuration achieved an inference time of 1.35ms, representing a 19.6% performance improvement over the host-only implementation of the quantised EMNIST CNN, the pure computational time of the kernels accounted for only 34% of the total wall time, as can be inferred from Table 5.3.

Table 5.3: Performance comparison between the 3D work-item and single work-item accelerator implementations, where the total wall time corresponds to the inference time for a single image.

Implementation	Total WallTime (ms)	Total Kernels Time (ms)	conv1 Time (µs)	mxpl1 Time (µs)	conv2 Time (µs)	mxpl2 Time (µs)	fc1 Time (µs)	fc2 Time (µs)
3D work-item	1.35	0.459	111.2	88.6	108.3	59.1	47.8	43.9
Single work-item	1.22	0.411	85.1	74.9	96.1	56.2	51.4	47.7

Profiling the 3D work-item accelerator revealed that performance was limited by lenghty stalls between successive kernel executions. These stalls were attributable to host control overheads and global memory transfers, as illustrated in Fig. 5.4. It should be noted, however, that using the '-profile' compilation flag introduces instrumentation hardware, which inflates kernel execution times. Nonetheless, the profiling results remain indicative of the design bottlenecks and control delays.

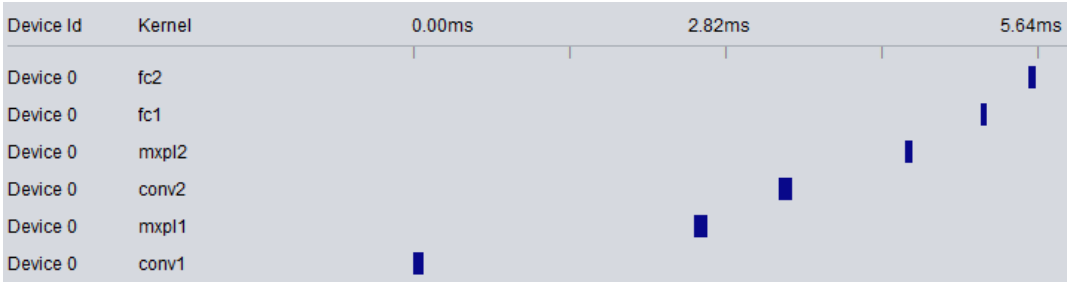


Figure 5.4: Screenshot of the Profiler GUI for the 3D work-item accelerator. The figure highlights long stall periods between short kernel execution intervals. The total wall time appears inflated from 1.35 ms to 5.64 ms due to the added profiling instrumentation.

Consequently, optimisation efforts focused on mitigating these control-induced stalls. Insights from Intel's Best Practices Guide indicated that FPGAs achieve higher performance when kernels are implemented as pipelined single work-item designs, rather than parallel 'NDRange' models. Furthermore, in pipelined loops, the compiler schedules new loop iterations at regular intervals, known as the *initiation interval* (II). An II of 1 enables launching a new iteration every clock cycle, as illustrated in Fig. 5.5, thereby maximising throughput after pipeline ramp-up [27, pp. 10, 66]. In contrast, multi-work-item designs incur area overhead and suffer from additional stall cycles due to synchronisation and work-group scheduling requirements.

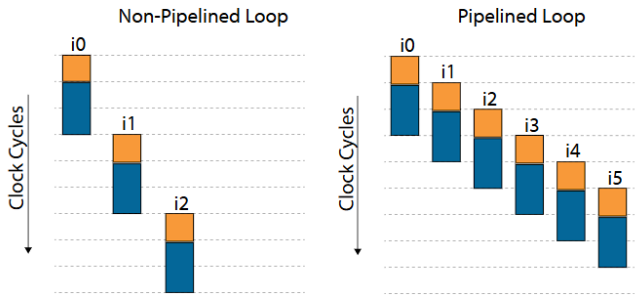


Figure 5.5: Comparison of non-pipelined and pipelined loops, showing an initiation interval of 1 for the pipelined one.

Therefore, the kernels were redesigned as single work-item implementations, guided by an Altera FIR filter design example [28], which was used solely to understand kernel structuring techniques. This example informed the restructuring of the design to process the entire dataset sequentially without relying on 'get_global_id()'. Additionally, the *max_global_work_dim(0)* attribute was applied to enforce pipelined hardware generation.

Moreover, FIFO channels shown in Listing 5.3 were implemented to stream intermediate feature maps directly between kernels, thereby eliminating costly global memory transfers. Besides the channels, BRAM units were used to cache these feature maps, enabling parallel MAC operations. As a result, the accelerator achieved a reduced inference time of 1.22 ms per image, as demonstrated in Table 5.3.

Kernels.cl

```
#pragma OPENCL EXTENSION cl_intel_channels : enable

channel signed char conv1_out_ch __attribute__((depth(BATCH*MXPL1_IN_SZ*MXPL1_IN_SZ)));
channel signed char mxpl1_out_ch __attribute__((depth(BATCH*MXPL1_OUT_SZ*MXPL1_OUT_SZ)));
channel signed char conv2_out_ch __attribute__((depth(BATCH*CONV2_OUT_IM_SZ*CONV2_OUT_IM_SZ)));
channel signed char mxpl2_out_ch __attribute__((depth(BATCH*FC1_IN_SZ)));
channel signed char fc1_out_ch   __attribute__((depth(BATCH*FC2_IN_SZ)));
```

Listing 5.3: Declaration of FIFO channels in the kernels '.cl' file. Each channel is 8 bits wide and parametrised with a depth equal to the total feature maps size for the entire batch.

Although the implementation of a single work-item design with channels resulted in a modest reduction in individual kernel execution times, as seen in Table 5.3, the total wall time was not substantially improved. For example, the execution time of conv1 and conv2 decreased by 26.1 µs and 12.2 µs respectively. However, the proportion of total kernel computational time relative to wall time remained around 34% for both 3D work-item and the single work-item designs. This indicates that overhead from kernel launches continued to dominate. To address this, the autorun feature was employed, informed by Intel's Autorun Kernel design example [29]. The autorun attribute instructs the hardware compiler to generate a dedicated hardware scheduler, enabling kernels to launch automatically without the control overhead required between the host and the FPGA. As a result, the wall time was nearly halved, decreasing from 1.22 ms to 0.631 ms.

While the autorun feature significantly improved the performance, the overall computational load remained low, as only a single image was processed per kernel launch. An OpenCL tutorial video [30] suggested amortising kernel launches and resource initialisation overhead with heavier workload, which inspired the use of batch processing to prolong kernels computations. Henceforth, multiple batch sizes were assessed, where using bigger batches required reducing the unrolling factors in the fully-connected layers loops. This was necessary because the limited BRAM resources caused the compiler to report errors once memory utilisation exceeded available capacity. Figure 5.6 below shows how batch processing effectively reduced inference time per image, where a batch size of 12 was found to yield the best performance. It should be noted, however, that these results were obtained with SoftMax computations still integrated within the 'FC2' kernel.

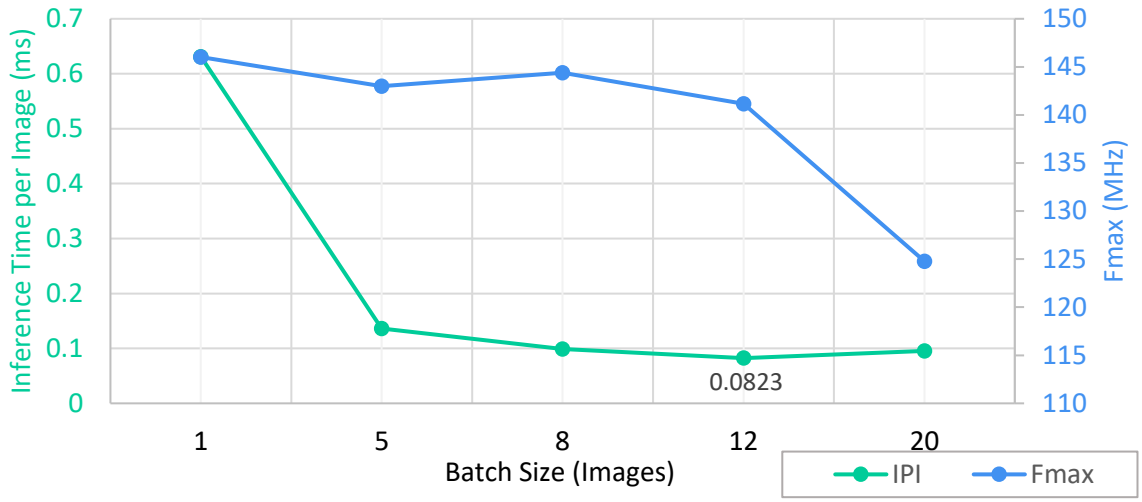


Figure 5.6: Performance comparison across increasing batch sizes. The green line show inference time per image, demonstrating over six orders of magnitude improvement compared to single-image processing. The blue line shows the corresponding maximum kernel clock frequency decreasing as batch size increases due to increased fanout.

The final round of optimisations addressed a bottleneck caused by the exponential floating-point operations of the SoftMax layer. The compilation report's system viewer, seen in Fig. 5.7, highlighted that the exponential computations loop imposed an initiation interval of 16 as opposed to the optimal value of 1. Consequently, a simpler and more efficient solution was devised by using the index of the neuron with the highest activation in the final fully connected layer as the predicted class label, effectively implementing an **Argmax** operation. This substitution works because the accelerator is intended for forward propagation only, where differentiability of the output function is not required. While SoftMax was used during training due to its differentiability, at inference time Argmax suffices, enabling the elimination of costly exponential computations.

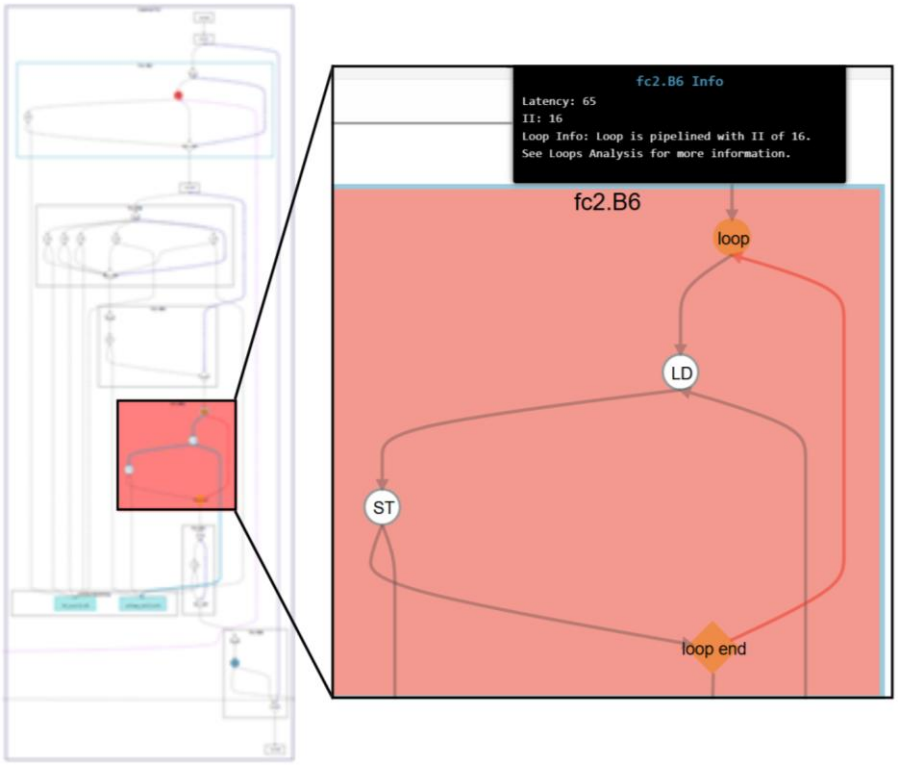


Figure 5.7: Screenshot of the compilation report's system viewer, highlighting a bottleneck caused by an initiation interval of 16 in the loop corresponding to the floating-point exponential computations of the SoftMax function.

The implementation of Argmax reduced hardware resource usage by eliminating all floating-point operations in the final prediction stage. As a result, additional resources were freed, allowing the batch size to be increased from 12 to 16 images. Furthermore, this optimisation reduced the initiation interval of the final kernel from 16 to 1, enabling a new iteration to begin every clock cycle. Additional minor optimisations included prefetching input images from the host into local BRAM and applying loop coalescing, which merges nested loops to reduce loop control overhead. However, these modifications had only a marginal impact, resulting in slight reductions in hardware usage without improving overall performance. The combination of these optimisations yielded the best inference time per image of 75.6 µs, representing an 8% improvement over the previously best-performing design with a batch size of 12.

6 Testing and Results Evaluation

6.1 Testing methodology

To assist the verification of the accelerator's computation algorithms and given that OpenCL syntax aligns with C99 syntax, C++ implementations of the CNNs were developed to emulate forward propagation of the trained networks. These implementations also served as host-only baseline benchmarks, enabling performance comparisons between the accelerator and ARM host-only execution. The ARM host performance was measured by recording the total time to classify 10,400 images from the EMNIST letters dataset, then dividing this time by the number of images. Dataset pre-quantisation computations were excluded from the timing to match the accelerator's measurement method, which assumes inputs are readily-quantised.

In the initial stages of kernels development, when intermediate data was still transferred back to the host, their functionality was individually verified by comparing the output feature maps against reference results obtained from MATLAB. For example, Figure 6.1 shows the output of a floating-point implementation of the 'Convolution1' kernel, where the results were validated to be accurate up to the fifth decimal point. This verification also confirmed the correct extraction and storage of weights and biases, particularly given that MATLAB stored matrices in column-major order.

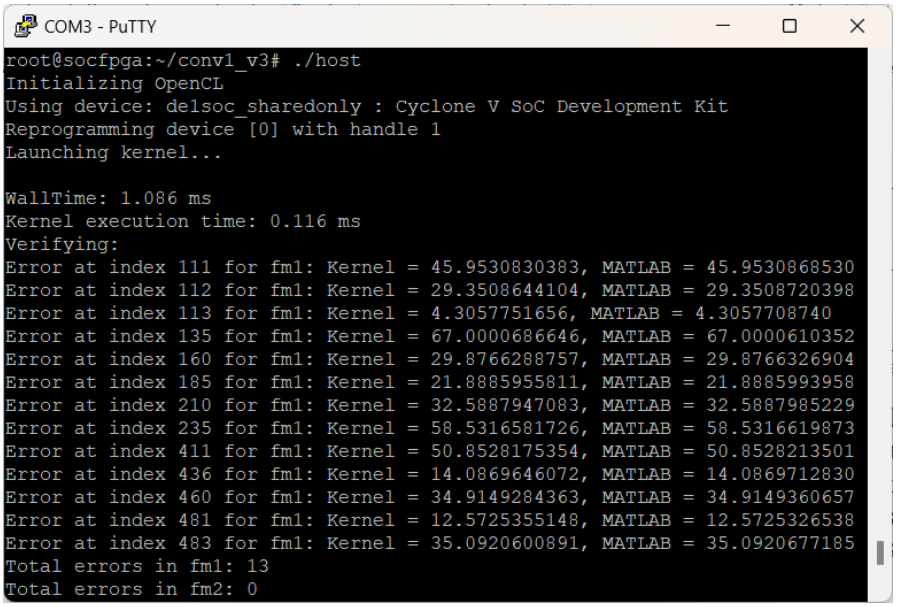


Figure 6.1: 'Convolution 1' kernel floating-point early implementation, showing correct operation up to 5 decimal points.

As hardware optimisations were introduced incrementally, each change was applied in isolation to allow for a clear evaluation of its impact. Moreover, performance evaluation of individual kernels was performed using the 'clGetEventProfilingInfo' function in the host application, which returns the execution time of kernel events. Additionally, to identify bottlenecks, the OpenCL profiler GUI was used, as discussed in the previous chapter. Most importantly, the '-report' compilation flag was consistently enabled across all builds to generate detailed estimates of resource usage and area analysis, including per-line hardware utilisation of the kernels. Moreover, the loop analysis tab in the generated report was used to examine the initiation intervals of critical loops. Combined with the

system viewer, this enabled tracing of performance bottlenecks, such as in the SoftMax layer case illustrated in Fig. 5.7.

After all kernels were integrated, the accelerator's functional correctness was further validated by running inference on multiple images to ensure consistent and accurate predictions. To benchmark the best-performing design, the host application was modified to loop through a sample of 10,400 images from the EMNIST letters test dataset, which were stored in a header file alongside their labels. Only half of the test dataset was used due to memory limitations encountered when compiling the updated host application. The benchmarking process evaluated classification accuracy, average inference time per image, frame rate (FPS), and overall accelerator throughput. Figure 6.2 presents the benchmarking results of the final design, where the equations used to compute these metrics are provided in Appendix B.

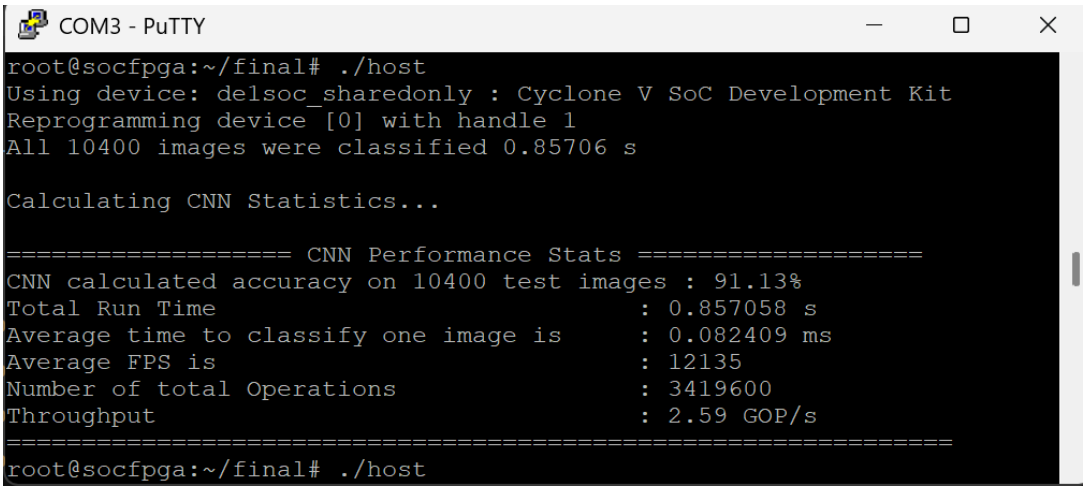


Figure 6.2: Screenshot of the benchmarking results of the final accelerator design (V7).

Additionally, it should be noted that the accelerator's performance exhibited a slight jitter between runs, as with any scheduled application, despite the host application being the only user process executed by the Linux OS. Therefore, all reported execution time values were represented as the average of 10 runs of the host application.

6.2 Results Comparison

To provide comparative evaluation of the final design performance, one recent and methodologically similar work [4] was selected for direct comparison. This paper presents a compiler-driven workflow in which CNN models are converted into OpenCL kernels using the TVM (Tensor Virtual Machine) framework. Their approach introduces various automated optimisations, including loop unrolling, loop fusion, cached writes, optimised floating-point operations, channelisation, autorun kernels, concurrent execution, loop tiling, and parameterised kernels.

By contrast, this project implemented most of these optimisation techniques manually. For instance, using loop coalescing pragma is similar the paper's loop fusion. Likewise, BRAM was employed to cache input images and enable reuse of intermediate feature maps passed through FIFO channels, which is comparable to the paper's use of cached write buffers. However, techniques such as concurrent kernel execution were not adopted here. Although OpenCL permits the use of multiple command queues to enable parallel kernel launches, this design employed a single queue and instantiated only one compute unit per kernel. This choice was made to maintain simplicity and given the available

resources did not allow for multiple compute units instantiation. Moreover, unlike the referenced paper, this work employed batch processing, which achieved 7.7× speedup when increasing the batch size from a single image to 12.

In terms of workload complexity, the referenced paper evaluates a LeNet-5-based accelerator targeting the MNIST dataset, which has 10 output classes. In contrast, the accelerator presented in this work manages the more demanding EMNIST letters dataset, which comprises 26 classes. However, unlike this project, the TVM-based approach does not apply any optimisation to the CNN model itself. As a result, the paper's accelerator has a higher computational workload, while this implementation achieves higher efficiency under tighter resource constraints through the reduction in learnable parameters and the INT8 quantisation.

In terms of throughput, [4] reports a performance of 3.49 GFLOPS based on an assumed 2.29M floating-point operations. In contrast, the proposed design achieves 2.59 GOPS over an estimated 3.4M operations, comprising integer MACs, fixed-point re-quantisation operations, ReLU activations, and pooling comparisons. Hence, it is plausible that if [4] adopted quantisation and batch processing, their design would surpass the present design.

To compare resource usage and performance, Table 6.1 shows that the proposed design achieves a 2.47× speedup in FPS compared to the referenced paper, while using significantly fewer hardware resources. Notably, the reference implementation in [4] targets a PCIe D5005 Programmable Acceleration Card (PAC) with a Stratix 10SX FPGA, which is a much more powerful and expensive platform than the used Cyclone V. Hence, to provide a fair comparison of hardware utilisation and design maximum frequency, the final design in this work was also compiled for the DE10-Pro development board, which hosts a Stratix 10SX FPGA with the same core resources as the D5005 PAC. Moreover, the compilation for the DE10-Pro used the 'Pro' version of Quartus prime and OpenCL SDK similar to [4].

Table 6.1: Comparison of resource usage, maximum kernel clock frequency (Fmax), and FPS between the proposed accelerator and the implementation in [4]. Resource values in the first row are inferred from reported utilisation percentages and the known specifications Stratix 10SX used in the D5005 PAC.

Accelerator	FPGA	Logic	BRAM	DSP	Fmax (MHz)	Dataset	FPS
S. Chung et. al [4] (D5005 PAC)	Stratix 10 SX (1SX280HN2F43E2VG)	700,000	2,227	576	218	EMNIST	4,917
This work (DE1-SoC)	Cyclone V (5CSEMA5F31C6N)	11,570	397	87	134.58	MNIST	12,135
This work (DE10-Pro)	Stratix 10 SX (1SX280HU2F50E1VG)	89,191	649	56	293.75	EMNIST	N/A

The results, summarised in Table 6.1, demonstrate that the proposed kernel architecture delivers a higher design clock frequency and lower resource consumption when mapped to the same FPGA. While the DE10-Pro board was not available, it is reasonable to infer that, given the increased logic and memory resources, further performance gains would be possible by enabling the instantiation of multiple kernel compute units and applying higher unrolling factors.

6.3 Critical Evaluation

The primary and stretch goals of this project were successfully achieved; however, minor adjustments were made to the stretch goals. Initially, the plan to enhance the CNN for finger vein map classification was replaced with EMNIST letters to enable standardised benchmarking. Additionally, CNN pruning was originally considered, but it was excluded in favour of focusing on hardware architecture optimisations.

The evaluation of the optimisations against the inference time on the ARM host alone is shown in Table 6.2, which also illustrates the evolution of performance across the optimisation stages. The WallTime refers to the time taken from launching the CNN functionality to receiving the final output predictions. For a batch size of one, WallTime equals the time per image, and the reported speedup is calculated as the ratio of the host-only baseline time per image to the accelerator time per image.

Table 6.2:Comparison summary of the quantised accelerator versions against the ARM host performance on the EMNIST CNN.

Accelerator	WallTime (ms)	Time per Image(ms)	Speedup	Fmax (MHz)	Highest fanout
Arm host Implementation	-	1.6750	-	-	-
V1: 3D Work-item	1.346	1.3460	1.24×	131.09	773
V2: Single Work-item with Channels	1.22194	1.2219	1.37×	145.62	947
V3: Autorun	0.63125	0.6313	2.65×	146.02	960
V4: Processing Batch of 12 Images	0.988	0.0823	20.34×	141.16	4697
V5: Cashing Input Images	0.997	0.0831	20.16×	140.29	5079
V6: Loop Coalescing	0.99	0.0825	20.3×	140.05	4882
V8: Argmax, Batch of 16 Images	1.21	0.0756	22.14×	134.58	5407

Aside from the transition from 3D to single work-item (V1 to V2), all subsequent versions in Table 6.2 build cumulatively on previous ones. Initial improvements from adopting single work-item kernels and using autorun scheduling reduced WallTime from 1.346 ms to 0.631 ms. This move to a pipelined design, with no additional host-side control delays, resulted in the highest kernel clock frequency of 146 MHz and the shortest WallTime (V3). However, the most significant gains were realised with batch processing in V4. Despite a slight increase in WallTime, the V4 optimisation achieved the highest improvement in speedup compared to the host implementation, as the fixed costs of kernel launches were amortised by the larger computational output. Subsequent refinements, including input caching (V5) and loop coalescing (V6), did not yield further speedup but led to a slight reduction in ALUT and register utilisation, as shown in Fig. 6.3. This suggests that these techniques could be omitted unless power efficiency is critical, in which case the reduced hardware usage would help limit the design's overall power consumption, especially if applied on a larger scale than this work.

Although Fmax varied slightly across versions, these changes demonstrate that architectural restructuring was the primary driver of performance improvements, rather than the operating frequency itself. The increase in fanout, particularly from V4 onwards, reflects the increased critical path introduced by higher batching and deeper pipelining.

While the presented speedup values were averaged over 10 runs of the application, each run launched the kernels only once. The final design (V7) achieved a 22.14× speedup over the host baseline. In contrast, the results shown in Fig. 6.2 were obtained by launching the V7 application once and executing the kernels in a loop of 650 iterations, each processing a batch of 16 images to classify the 10,400 test samples. This setup led to an average time per image of 0.082 ms, due to increased backlog effects in buffer management under

sustained operation. Nevertheless, the speedup of the final design under these conditions remains high at 20.3× relative to the host-only baseline.

Figure 6.3 presents the evolution of FPGA resource utilisation across the quantised EMNIST CNN accelerator iterations. The figure shows how DSPs and BRAM usage reached 100% from V4 onward, which constrained further loop unrolling, or larger batching. In contrast, ALUTs and register utilisation remained moderate throughout and showed a consistent decline in the later stages, particularly after the adoption of Argmax in V7. This drop can be attributed to the elimination of the SoftMax layer, which previously involved costly floating-point exponentiation. Similarly, overall logic utilisation decreased from 71% in V2 to just 36% in the final design. Moreover, memory usage initially dropped with the introduction of the FIFO channels (V2) but increased again after introducing batching, as larger intermediate data buffers were required.

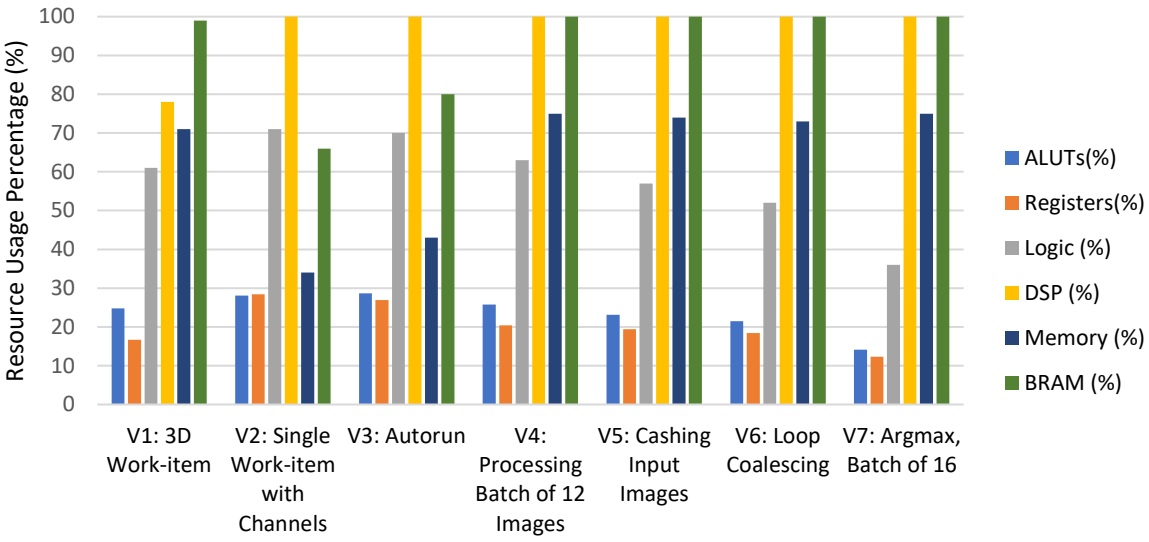


Figure 6.3: Comparison of resource usage across the different accelerator versions.

Despite the achieved performance improvements, the final design exhibited inherent limitations. Full utilisation of DSP and BRAM resources from V4 onward restricted further kernel replication and batch size increases. Moreover, while memory read and write stalls were mostly eliminated, stalls were still observed on channel reads and writes according to the System Viewer report. These stalls are likely caused by minor timing mismatches between producer and consumer kernels.

To further explore potential architectural optimisations under the observed resource constraints, an experimental design was evaluated in which all CNN layers were merged into a single kernel. The motivation for this approach was to eliminate the need for inter-kernel communication via channels and to reduce control flow overhead, thereby potentially freeing additional hardware resources. It was anticipated that this could enable a larger batch size or allow more loop unrolling, thus increasing the degree of parallelism. However, experimental results showed that this implementation did not achieve a higher inference performance, nor did the compiler significantly reduce resource utilisation as expected. The reason for this could be that merging kernels increases logic depth, and critical paths, reducing clock speed and pipelining efficiency. In contrast, when separate layers are made into kernels connected by channels, the compiler can overlap the execution of different kernels in a better pipeline.

6.4 Design Implications for Deployment

While the FPGA implementation in this project serves primarily as a prototyping platform, in commercial large-scale deployments, application-specific integrated circuits (ASICs) are typically used due to their higher efficiency, greater customisability, and significantly larger on-chip memory. An ASIC version of this accelerator would eliminate many of the stalls observed from global memory transfers thanks to increased bandwidth and dedicated memory hierarchies. Furthermore, ASIC implementations typically offer significantly improved power efficiency compared to FPGAs, enabling deployment in energy-constrained environments.

In the context of real-time applications, using OS like Linux is known to introduce scheduling-related jitter, which results in less deterministic behaviour compared to dedicated ASIC processors running bare-metal firmware. However, since the system runs a minimal Linux environment without competing user processes, jitter is expected to be lower than on general-purpose systems such as Windows. While detailed characterisation of jitter was beyond the scope of this project, future work could assess the impact of the applied optimisations on system jitter.

7 Reflection

7.1 Effectiveness of High-Level Synthesis

High-level synthesis tools represent a recent advancement in digital hardware design, offering faster development cycles compared to traditional HDL-based approaches. These tools enable the design of increasingly complex hardware architectures with reduced prerequisite experience, thereby lowering barriers to entry in digital hardware development. Numerous research studies have demonstrated the successful application of HLS in designing accelerators for convolutional neural networks (CNNs), signal processing, among other applications [4], [9], [22]. Furthermore, debugging hardware generated via HLS is simplified due to a substantial reduction in code complexity, as developers can simply print outputs rather than manually inspecting individual data bus signals to verify the computations.

Despite these advantages, concerns emerged from practical experience with Intel's OpenCL SDK. Firstly, the Intel OpenCL compiler exhibits non-deterministic behaviour, meaning that compiling identical kernels can unpredictably yield varying hardware resource usage. Additionally, the compiler automatically promotes data types to a 32-bit width to align operands, even when originally defined as narrower types (e.g., 8-bit integers). This implicit widening occurs when smaller data types interact with larger ones, which increased resource utilisation. Moreover, experimental results showed that defining all counters uniformly as 32-bit integers yielded slightly better performance than defining them based on their maximum bounds and expected operand widths (using 8-bit or 16-bit types where appropriate), as shown in Table 7.1.

Table 7.1: Effect of using narrower datatypes for loop iteration variables on V4 implementation of the accelerator.

Implementation	WallTime (ms)	Time per Image (ms)	ALUTs	Registers	Logic (%)	Fmax (MHz)
V4	0.9880	0.0823	28196	44644	63	141.16
V4 (narrower data types)	1.0080	0.0840	27371	43118	60	136.87

In retrospect, Intel's OpenCL SDK has proven effective in accelerating CNN computations compared to general-purpose CPUs. However, achieving optimal performance requires meticulous attention to the SDK's documentation and features. For example, leveraging local memory does not inherently guarantee improved performance as evidenced by V5 in Table 6.2. Similarly, increased loop unrolling without careful analysis can degrade performance. Most crucially, kernel launches must process sufficiently large workloads to justify the overhead associated with global memory transfers and kernel initiation. Additionally, implementing single work-item kernels with pipeline architectures using channels and autorun kernels was essential to reduce stalls.

Finally, this project demonstrated that Intel's OpenCL SDK could be effectively self-taught within undergraduate-level projects given experience with C++. Furthermore, performance levels comparable to state-of-the-art research are achievable. Nevertheless, fundamental understanding of digital design principles remains essential to interpret the compiler's response to code alterations and comprehend the generated files.

7.2 Project Management

Since most aspects of this project involved new concepts such as CNN architectures and OpenCL, it required careful planning, risk management, and motivation. A Gantt chart was created, providing ample time for design validation and accommodating potential delays caused by debugging or technical difficulties. The schedule in the submitted Gantt chart was generally adhered to in the first semester as shown in Appendix C; however, delays occurred due to changing the design approach, selecting suitable HLS tools, and setting up the OpenCL environment. Despite these delays, time was recovered because development and debugging cycles in OpenCL are faster compared to HDL. Furthermore, as optimisations took place in the second semester, the project did not strictly adhere to the original timeline, as each design evaluation informed the next logical optimisation.

To reduce risks further, the complex task of designing the hardware accelerator was broken down into simpler, manageable blocks. Each newly developed block was validated against reference calculations performed in MATLAB. To manage the learning curve, development began with fully connected layers due to their simplicity, followed by the other layers. Additionally, due to the significant risk of resource exhaustion when using a complex CNN architecture, the development started with a simple binary classification task of ellipses, then gradually increased to more challenging classification tasks while maintaining the resource usage.

Project design changes were tracked with ‘README’ files explaining each version. Moreover, Excel sheets tracked performance metrics, while an MS Word document served as a digital logbook recording learned concepts, design rationale, testing plans, and notes from weekly supervisory meetings. Additionally, weekly backups of project files were stored on an external SSD to prevent data loss.

8 Conclusion

8.1 Achievement Summary

This project successfully implemented an FPGA-based quantised accelerator for forward propagation of a LeNet-5-based CNN, specifically designed for classifying the EMNIST letters dataset. The design, developed using Intel's OpenCL SDK, features a fully pipelined streaming architecture composed of single work-item kernels, FIFO channels, autorun scheduling, and batch processing to amortise kernel launch overheads. The final implementation achieved a speed of 12,135 images per second, surpassing the performance reported in a recent paper [4], while using significantly fewer hardware resources. Finally, this work offers practical insights into the design and optimisation of FPGA-based accelerators using OpenCL.

While the CNN operations used are standard, this work demonstrates originality by replacing the SoftMax layer with an embedded Argmax functionality directly within the FC2 neurons loop, an approach not explicitly reported in prior works. Furthermore, most prior works target high-end FPGAs and fit larger CNNs fully on-chip, while others that use the same DE1-SoC platform [22] accelerate only the convolution layers, relying on the ARM host for the remainder of the network due to resource limitations. In contrast, this work fits the entire CNN onto the resource-constrained FPGA by using a simpler CNN architecture. Nevertheless, the applied optimisation techniques are scalable and could be extended to more complex networks when targeting higher-resource FPGAs.

8.2 Future Work

Given access to a more advanced FPGA development kit with greater hardware resources, the effectiveness of the same optimisation techniques could be tested on more complex CNN architectures, such as MobileNet, to provide comparison to similar literature. Follow-up work could apply the lessons learned from optimising kernel design to other computationally demanding applications that would benefit from ad-hoc hardware architectures, such as post-quantum cryptography primitives, cryptographic hashing functions, or signal processing tasks like the Fast Fourier Transform algorithm. Additionally, further optimisation techniques described in Intel's OpenCL optimisation guide could be explored. These include using the RTL module feature for finer control over hardware resources and developing a multi-threaded host application to better exploit parallelism.

References

[1] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, 'Efficient Processing of Deep Neural Networks: A Tutorial and Survey', *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017, doi: 10.1109/JPROC.2017.2761740.

[2] 'Deploying Transformers on the Apple Neural Engine', Machine Learning Research at Apple. Accessed: Apr. 14, 2025. [Online]. Available: https://machinelearning.apple.com/research/neural-engine-transformers

[3] 'Intel Core Ultra Processors 200HX Series Processors - Quick Reference Guide', Intel.com. Accessed: Apr. 14, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/842532/intel-core-ultra-processors-200hx-series-processors-quick-reference-guide-pdf.html

[4] S. Chung and T. S. Abdelrahman, 'Optimization of Compiler-Generated OpenCL CNN Kernels and Runtime for FPGAs', in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, May 2022, pp. 100–103. doi: 10.1109/IPDPSW55747.2022.00026.

[5] Z. Yang, L. Lu, and R. Wang, 'A batched GEMM optimization framework for deep learning', *J. Supercomput.*, vol. 78, no. 11, pp. 13393–13408, Jul. 2022, doi: 10.1007/s11227-022-04336-3.

[6] M. Abdelfattah, 'L2: ML Hardware', Cornell Tech: ECE 5545: Machine Learning Hardware and Systems. Accessed: Nov. 25, 2024. [Online]. Available: https://abdelfattah-class.github.io/ece5545/

[7] K. Abdelouahab, M. Pelcat, J. Serot, and F. Berry, 'Accelerating CNN inference on FPGAs: A Survey', May 26, 2018, *arXiv*: arXiv:1806.01683. Accessed: Oct. 21, 2024. [Online]. Available: http://arxiv.org/abs/1806.01683

[8] 'NVIDIA H100 Tensor Core GPU Datasheet', NVIDIA. Accessed: Dec. 03, 2024. [Online]. Available: https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet

[9] A. Yang *et al.*, 'An OpenCL-Based FPGA Accelerator for Compressed YOLOv2', in *2019 International Conference on Field-Programmable Technology (ICFPT)*, Dec. 2019, pp. 235–238. doi: 10.1109/ICFPT47387.2019.00036.

[10] E. Nurvitadhi, R. D'Souza, and M. Won, 'Real Performance of FPGAs Tops GPUs in the Race to Accelerate AI'. Intel White Paper.

[11] H. Almorin, B. Le Gal, J. Crenne, C. Jego, and V. Kissel, 'High-throughput FFT architectures using HLS tools', in *2022 29th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, Oct. 2022, pp. 1–4. doi: 10.1109/ICECS202256217.2022.9970886.

[12] S. Li, Y. Luo, K. Sun, N. Yadav, and K. K. Choi, 'A Novel FPGA Accelerator Design for Real-Time and Ultra-Low Power Deep Convolutional Neural Networks Compared With Titan X GPU', *IEEE Access*, vol. 8, pp. 105455–105471, 2020, doi: 10.1109/ACCESS.2020.3000009.

[13] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, 'Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks', in *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Monterey California USA: ACM, Feb. 2015, pp. 161–170. doi: 10.1145/2684746.2689060.

[14] C. Yang, H. Zhang, X. Wang, and L. Geng, 'An Energy-Efficient and Flexible Accelerator based on Reconfigurable Computing for Multiple Deep Convolutional Neural Networks', in *2018 14th IEEE International Conference on Solid-State and*

Integrated Circuit Technology (ICSICT), Oct. 2018, pp. 1–3. doi: 10.1109/ICSICT.2018.8565823.
[15] H. Yu and S. Li, 'A Higher Performance Accelerator for Resource-Limited FPGA to Deploy Deeper Object Detection Networks', in *2022 IEEE 16th International Conference on Anti-counterfeiting, Security, and Identification (ASID)*, Dec. 2022, pp. 1–5. doi: 10.1109/ASID56930.2022.9995953.
[16] L. Cheng, Y. Gu, Q. Liu, L. Yang, C. Liu, and Y. Wang, 'Advancements in Accelerating Deep Neural Network Inference on AIoT Devices: A Survey', *IEEE Trans. Sustain. Comput.*, pp. 1–18, 2024, doi: 10.1109/TSUSC.2024.3353176.
[17] Y. Song, B. Wu, T. Yuan, and W. Liu, 'A High-Speed CNN Hardware Accelerator with Regular Pruning', in *2022 23rd International Symposium on Quality Electronic Design (ISQED)*, Apr. 2022, pp. 1–5. doi: 10.1109/ISQED54688.2022.9806216.
[18] V. H. Kim and K. K. Choi, 'A Reconfigurable CNN-Based Accelerator Design for Fast and Energy-Efficient Object Detection System on Mobile FPGA', *IEEE Access*, vol. 11, pp. 59438–59445, 2023, doi: 10.1109/ACCESS.2023.3285279.
[19] M. Kumar and G. Kaur, 'HPC Workflow on Diverse XPU Architectures with oneAPI', in *2022 2nd International Conference on Intelligent Technologies (CONIT)*, Jun. 2022, pp. 1–5. doi: 10.1109/CONIT55038.2022.9848296.
[20] K. Obata, H. M. Waidyasooriya, and M. Hariyama, 'Implementation of an FPGA-Oriented Complex Number Computation Library Using Intel OneAPI DPC++', in *2022 IEEE 65th International Midwest Symposium on Circuits and Systems (MWSCAS)*, Aug. 2022, pp. 1–4. doi: 10.1109/MWSCAS54063.2022.9859514.
[21] S. I. Venieris, A. Kouris, and C.-S. Bouganis, 'Toolflows for Mapping Convolutional Neural Networks on FPGAs: A Survey and Future Directions', *ACM Comput. Surv.*, vol. 51, no. 3, pp. 1–39, May 2019, doi: 10.1145/3186332.
[22] H.-T. Ngo, T.-T. Duong, M.-T. Tran, and T.-T. Dang, 'Application of Object Detection Algorithm on SoC-FPGA using OpenCL', in *2023 4th International Conference on Communications, Information, Electronic and Energy Systems (CIEES)*, Nov. 2023, pp. 1–4. doi: 10.1109/CIEES58940.2023.10378810.
[23] 'DE1-SoC OpenCL'. Accessed: Apr. 17, 2025. [Online]. Available: https://download.terasic.com/downloads/cd-rom/de1-soc/linux_BSP/OPENCL18.1/DE1-SoC_OpenCL_v05.pdf
[24] 'OpenCL Vector Addition Design Example', Intel. Accessed: Apr. 17, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/support/programmable/support-resources/design-examples/horizontal/vector-addition.html
[25] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, 'Gradient-based learning applied to document recognition', *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.
[26] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, 'A White Paper on Neural Network Quantization', Jun. 15, 2021, *arXiv*: arXiv:2106.08295. doi: 10.48550/arXiv.2106.08295.
[27] 'Intel FPGA SDK for OpenCL Pro Edition: Best Practices Guide'. Intel, Dec. 19, 2022. [Online]. Available: https://www.intel.com/content/www/us/en/docs/programmable/683521/22-4/eol.html
[28] 'Time-Domain Finite Impulse Response FIR Filter Design Example', Intel. Accessed: Apr. 21, 2025. [Online]. Available:

https://www.intel.com/content/www/us/en/support/programmable/support-resources/design-examples/horizontal/td-fir.html
[29] ‘OpenCL kernel autorun feature’. Accessed: Apr. 21, 2025. [Online]. Available: https://community.intel.com/t5/FPGA-Wiki/OpenCL-kernel-autorun-feature/ta-p/735763
[30] David Black-Schaffer, *OpenCL Performance Tips and Summary (10)*, (Apr. 06, 2016). Accessed: Apr. 21, 2025. [Online Video]. Available: https://www.youtube.com/watch?v=ITvpmvH2tkc
[31] ‘Convolutional Neural Network | Deep Learning | Developers Breach’. Accessed: Nov. 19, 2024. [Online]. Available: https://developersbreach.com/convolution-neural-network-deep-learning/
[32] B. K. Kalejahi, S. Meshgini, S. Danishvar, and S. Khorram, ‘Diagnosis of liver disease by computer- assisted imaging techniques: A literature review’, *Intell. Data Anal.*, vol. 26, no. 4, pp. 1097–1114, Jul. 2022, doi: 10.3233/IDA-216379.
[33] S. SHARMA, ‘Activation Functions in Neural Networks’, Medium. Accessed: Oct. 15, 2024. [Online]. Available: https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6
[34] Nabi Nabiyev and S. Malekzadeh, ‘Anomalous Sound Localization Estimation’, 2021, doi: 10.13140/RG.2.2.25949.95201.

Appendix A. CNNs Background

This Appendix provides background information on CNNs operation and general structure. From a functionality perspective, a CNN's main building blocks are the feature extraction block, the fully-connected layer block, and the probabilistic distribution block, as illustrated in Fig. A.1 [31] below.

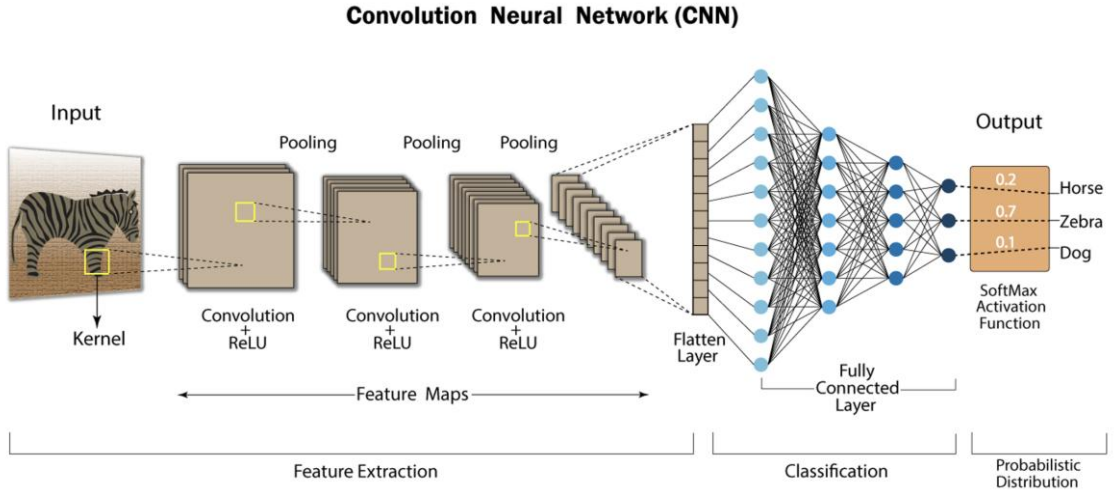


Figure A.1: Convolutional neural network structure showing the three main functional blocks, sourced from [31].

A.1 Feature Extraction Block

This block serves two primary functions: it performs convolutions to extract features from the input image and applies pooling to reduce computational complexity for subsequent stages.

Convolution is done through element-wise matrix multiplication of the input image, as seen in Fig. A.2 [32], having dimensions of H×W×C×B, with weight matrices called *filters*, and sized K×J×C. This multiplication is done iteratively as the filters slide across the input image to produce what is called an output *feature map* with the dimensions of U×V×N. Table A.1 shows the definitions of these parameters.

Table A.1: CNN's parameters definitions reproduced from [7].

Parameter	Definition
H	Height of the input image
W	Width of the input image
C	Number of channels in the input image
B	Batch size of the input image
K	Height (spatial dimension) of the filter
J	Width (spatial dimension) of the filter
U	Width of the output feature map
V	Height of the output feature map

The result of each multiplication is summed across all the channels to produce a pixel on the output feature map. This process can be described using equation (A1), where a *bias* term $\beta^{conv}[n]$ is added to control the resulting pixel activation [1]:

$$Y^{conv}[b,n,v,u] = \beta^{conv}[n] + \sum_{c=1}^{C}\sum_{j=1}^{J}\sum_{k=1}^{K} X^{conv}[b,c,v+j,u+k]\Theta^{conv}[n,c,j,k] \quad \text{(A1)}$$

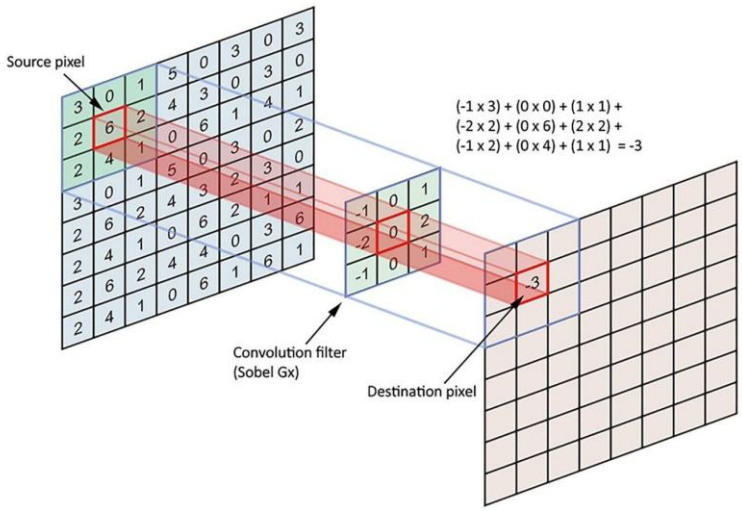


Figure A.2: Convolution process example showing element-wise matrix multiplication, sourced from [32],

The feature map size will be smaller than the input image due to the weighted sum during the convolution and will depend on the *stride* size S. The stride, as seen in Fig. A.3, is the amount of input image pixels the filter will skip to perform the next convolution. However, images could be padded with extra pixels, typically with the value of zero, to maintain the output feature map size.

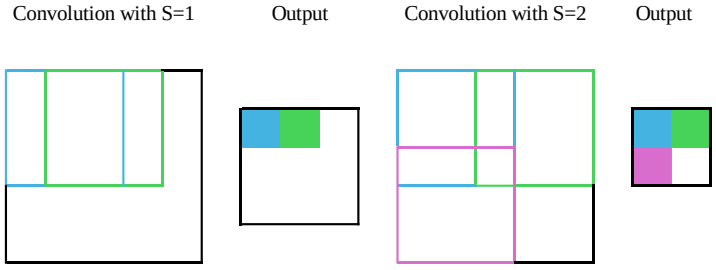


Figure A.3: Effect of changing stride size on the output feature map, reproduced from [1].

Pooling layers, often done after a convolutional layer, compress the image to a smaller one by taking the average or maximum value of a pixel from a pooling window as illustrated in Fig. A.4. This allows for the use of more layers in a network due to reduced computation.

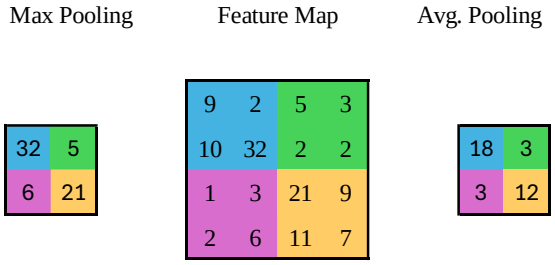


Figure A.4: Maximum and Average Pooling process example with a 2×2 pooling window, reproduced from [1].

A.2 Fully-Connected Layer Block

Once the feature maps are extracted, they are flattened from spatial tensors (multi-dimensional arrays representing spatial data) to a single vector, which is fed to the fully-connected layer to learn the extracted features. The fully-connected layer consists of neurons, each of which performs a weighted sum of the flattened vector with learned

weights, then adds a bias term sum to control how high the sum must be before the neuron starts to activate as illustrated in equation (A2).

$$a = f(\sum_{i=1}^{n}(w_i . x_i) + b) \quad \text{(A2)}$$

Where a is the output of the neuron after applying the activation function, f is the activation function, w_i are the weights, x_i are the input activations, and b is the bias term. The activation function normalises the weighted sum to a specific range, which introduces non-linearity to reduce the computational complexity and learning time. Fig. A.5 below illustrates commonly used activation functions [1], [33].

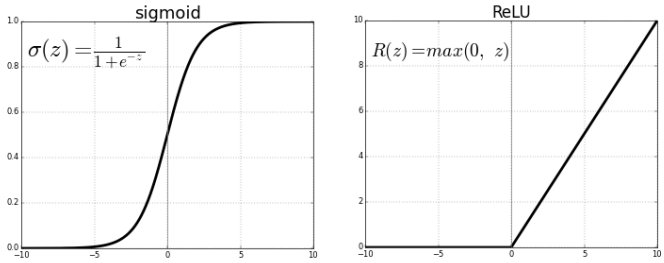


Figure A.5: Sigmoid and ReLU Non-linear activation functions illustration and their respective formulas, sourced from [33].

The weights and biases across the network are *learned* to obtain the correct classification output. This process boils down to finding the minimum in a *cost function*, which quantifies the error between the predicted output and the actual target.

A.3 Probabilistic Distribution Block

The final activation function takes the raw outputs from the final layer and converts them into probabilities that sum to one. Each output represents the model's confidence that a given input belongs to a particular class. The SoftMax function, illustrated in Fig. A.6 [34], is commonly used to perform this functionality.

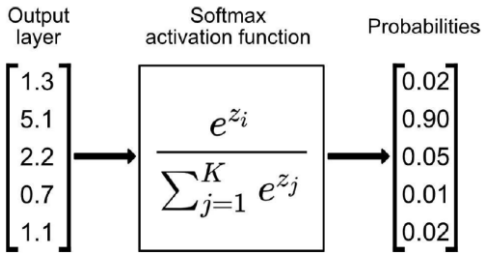


Figure A.6: SoftMax function operation ensures the sum of the predictions is equal to one, sourced from [34].

Appendix B. Benchmark Equations

The computational workload and throughput were estimated using the operation counts shown in Table B.1.

Table B.1: Expressions used to calculate the operations count and throughput.

Counted Value	Equation
Convolution MACs	$(CONV\_SZ)^2 \times (CONV\_OUT\_IM\_SZ)^2 \times CONV\_OUT\_CHAN \times CONV\_IN\_CHAN \times BATCH$
Convolution Re-quantisation	$(CONV\_OUT\_IM\_SZ)^2 \times CONV\_OUT\_CHAN \times BATCH$
Convolution ReLU	$(CONV\_OUT\_IM\_SZ)^2 \times CONV\_OUT\_CHAN \times BATCH$
Maxpooling comparisons	$CONV\_OUT\_CHAN \times (MXPL\_OUT\_SZ)^2 \times ((MXPL\_SZ)^2 - 1) \times BATCH$
FC layer MACs	$FC\_OUT\_SZ \times FC\_IN\_SZ \times BATCH$
FC layer Re-quantisation	$FC\_OUT\_SZ \times BATCH$
Argmax Comparisons	$(FC\_OUT\_SZ - 1) \times BATCH$
Average time of a Batch	$(\frac{total\_time}{DATASET\_SIZE}) \times BATCH$
Throughput (GOP/s)	$\frac{Total\ Operations}{\text{Average time of a Batch}}$

The definition of the variables shown in Table B.1 are clarified in Table B.2 below.

Table B.2: CNN Parameters and Their Values for Workload Estimation.

Variable	Definition	Value	
CONV_SZ	Convolution kernel size	Conv1=3	Conv2=5
CONV_OUT_IM_SZ	Output feature map width after Conv.	Conv1=26	Conv2=9
CONV_OUT_CHAN	Number of Conv. output channels	Conv1=5	Conv2=16
CONV_IN_CHAN	Number of Conv. input channels	Conv1=1	Conv2=5
MXPL_SZ	Maxpool kernel size	MXPL1=2	MXPL2=2
MXPL_OUT_SZ	Output size after Maxpool	MXPL1=13	MXPL2=4
FC_IN_SZ	FC input size	FC1 =256	FC2=30
FC_OUT_SZ	FC output size (number of neurons)	FC1 =30	FC2=26
BATCH	Number of images per batch	16	

Appendix C. Gantt Charts

Figure C.1 Shows the original plan for Semester 1.

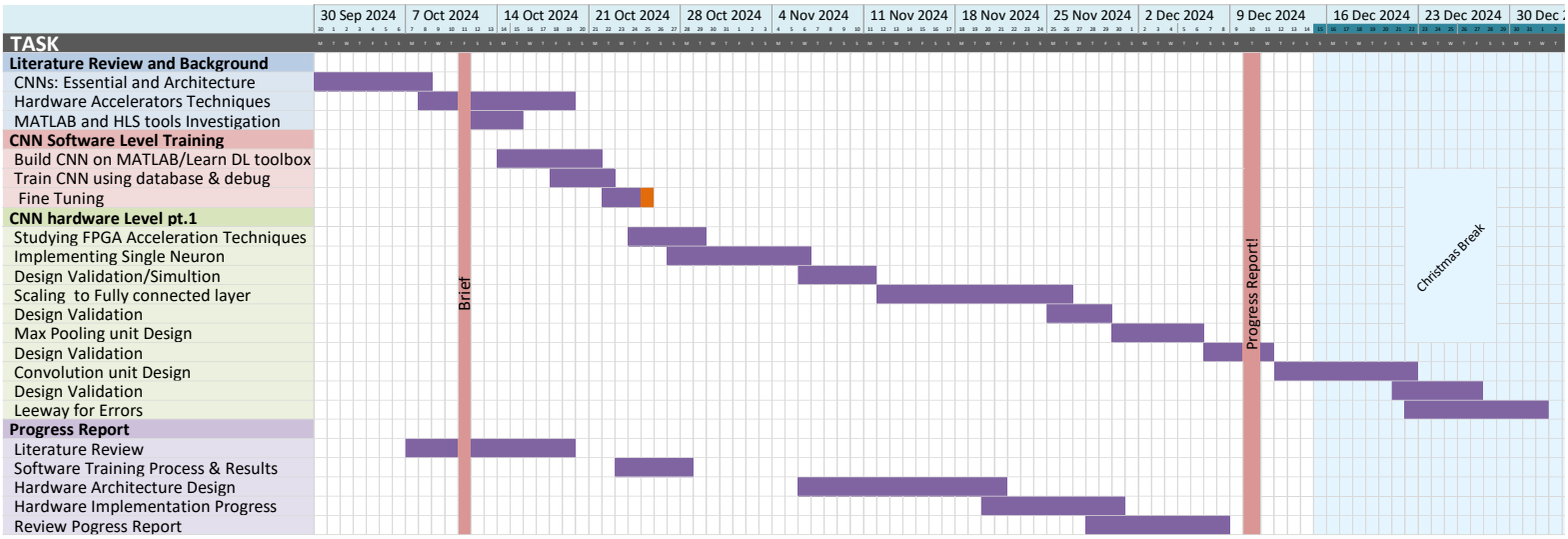


Figure C.1: Planned tasks for the first semester.

Figure C.2 shows the completed Part of the Gantt Chart in the first semester.

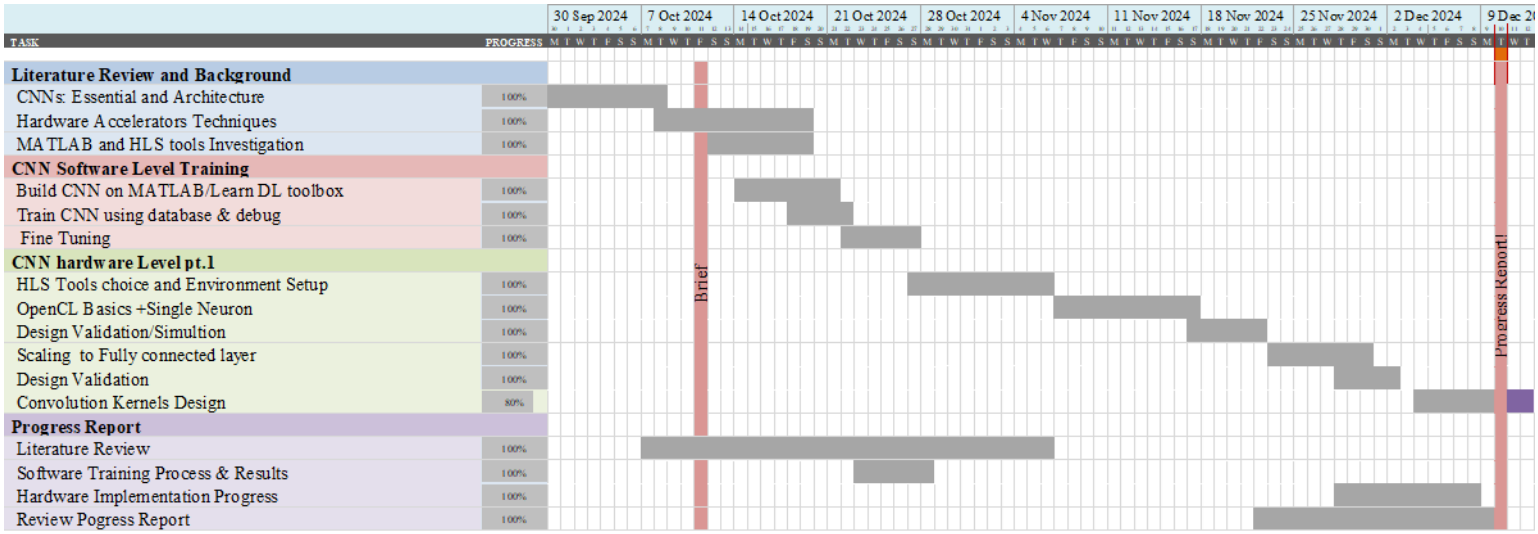


Figure C.2: Completed Gantt chart in the first semester

At the time of submission of the interim review, the plan for the tasks in the remaining of the first semester was as shown in Fig. C.3. However, no work took place during the winter break. Nevertheless, the convolutional kernels design was completed by the end of the first term, along with a host application that performed the rest of the inference functionality.

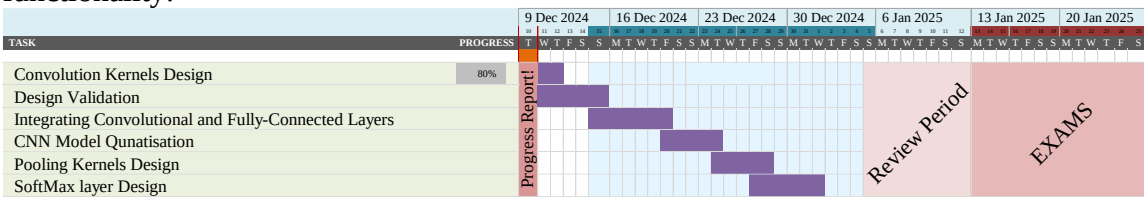


Figure C.3:Updated plan for the reminder of the first semester.

Figure C.4 shows the planned work for semester 2.

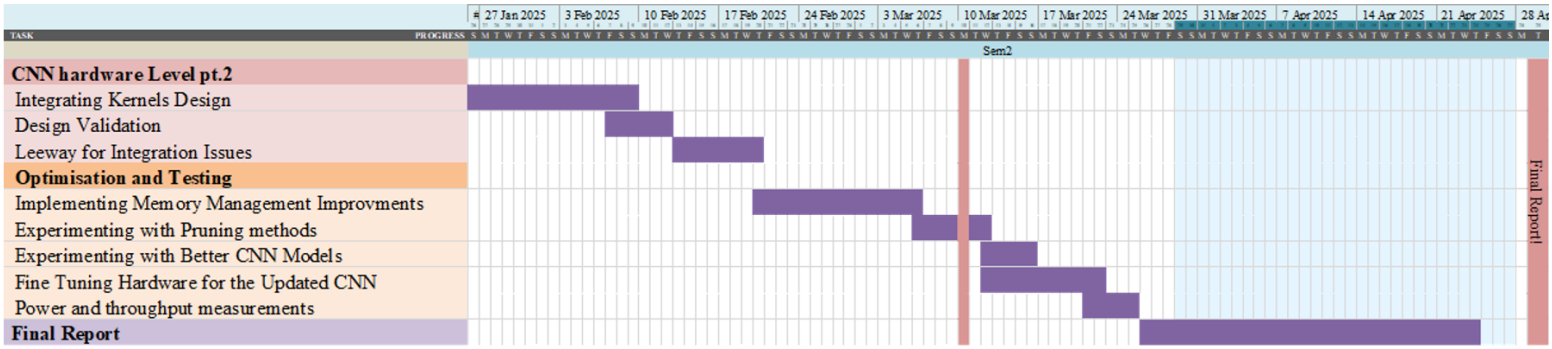


Figure C.4: Planned tasks for the second semester.

Figure C.5 below shows the actual work done for the second semester.

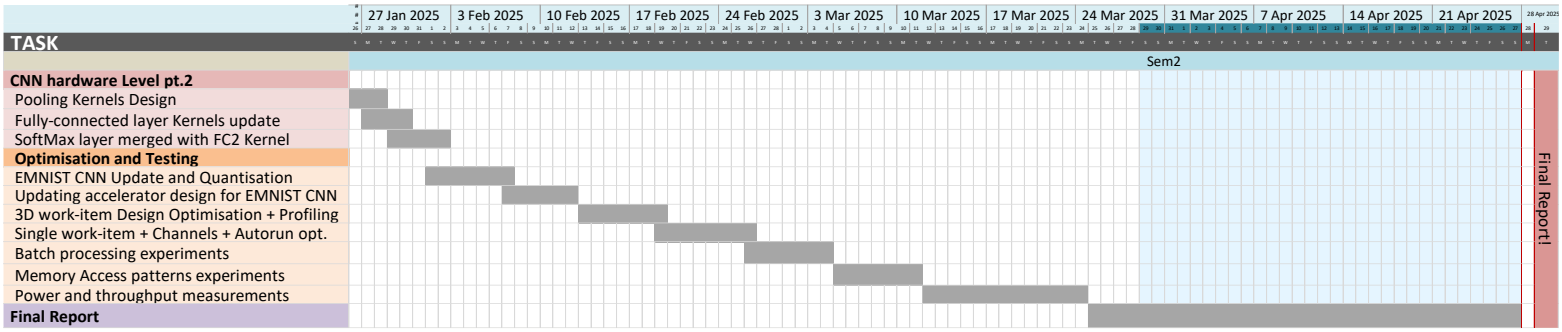


Figure C.5: Completed Gantt chart in the second semester

Appendix D. Design and Data Archive

The top-level folders are:

- CPP_code
- MATLAB_code
- ONNX_code
- OpenCL_code

1. CPP_code Folder Contents:

- Ellipses_CNN Folder:

Floating-point implementation of Ellipses CNN in C++.

- EMNIST_CNN:
    - EMNIST_CNN_float Folder:

    Contains floating-point C++ implementation of the EMNIST CNN using MATLAB exported parameters.
    - EMNIST_CNN_quantised Folder:

    Quantised C++ implementation classifying half the EMNIST test dataset (10,400 images).
    - manual_params_quantiser Folder:

    Scripts to quantise EMNIST CNN parameters using scales and zero-points from ONNX, and to quantise test images.

2. MATLAB_code Folder Contents:

- EMNIST_CNN.mlx:
  MATLAB script for training and testing EMNIST CNN accuracy.
- trainedNet_90.27Acc_0.3139Loss_10582Learnables.mat:
  Saved trained EMNIST CNN model.
- EMNIST_CNN.onnx:
  Exported ONNX model of the saved CNN.
- EMNIST_Sample_<letter>.png:
  Sample images exported from the EMNIST Letters dataset.
- emnist-letters ubyte files:
  Downloaded EMNIST Letters dataset files.
- params_exporter.mlx:
  Script exporting floating-point parameters and reference activations.

3. ONNX_code Folder Contents:

- ONNX_Quantiser.ipynb:
  Jupyter notebook with ONNX Runtime code for EMNIST CNN quantisation.
- emnist-letters ubyte files:
  Downloaded EMNIST Letters dataset files.
- EMNIST_CNN.onnx:
  The model exported from MATLAB to ONNX for quantisation.
- quantised_onnx_model.onnx:
  The output of ONNX quantisation.
- quantised_params.h:
  Exported quantised parameters

4. OpenCL_code Folder Contents:

- DE1_SoC_OpenCL_code:
  Design folders for accelerator versions V1–V7 and the ARM host-only baseline.
- DE10_pro_OpenCL_code:
  Compilation of the final V7 design for the DE10-Pro board.

Inside any OpenCL project, the folder structure is:

- device/: Contains kernel design (conv.cl).
- host/: Contains host application (main.cpp).
- bin/: Contains generated compilation files.

The key generated files to observe in the bin directory are below

- bin/conv/reports/: HTML report containing resource usage estimate.
- bin/conv/acl_quartus_report.txt: Actual FPGA resource usage report.
- bin/conv.aocx: FPGA bitstream.
- bin/host: Compiled host executable.

Appendix E. Using Intel's FPGA SDK for OpenCL

To use OpenCL on the DE1-SoC board, Terasic offers 'DE1-SoC OpenCL User Manual' that can be downloaded at their website. The manual details the process of setting up the environment for using Intel FPGA SDK for OpenCL.

The following software should be installed on the user's computer:
- Intel Quartus Prime Standard Edition 18.1.
- Intel FPGA SDK for Open CL Prime Edition 18.1.
- Intel SoC EDS 18.1.
- Win32 Disk Imager.
- PuTTY.
- WinSCP.

Additionally, install the board support package (BSP) from Terasic's website, which includes the Linux image file 'de1_soc_opencl.img'. Flash this image onto a microSD card using Win32 Disk Imager. Then, configure environment variables as per section 2.3 of the manual.

To prepare the board setup, follow the instructions below:
- Connect the UART to USB port (J4) to the computer.
- Use PuTTY with 115200 baud rate and serial communication.
- Set DIP switch (SW10) to [4:0] = 01010, Fig. E.1 below depicts the switch configuration.

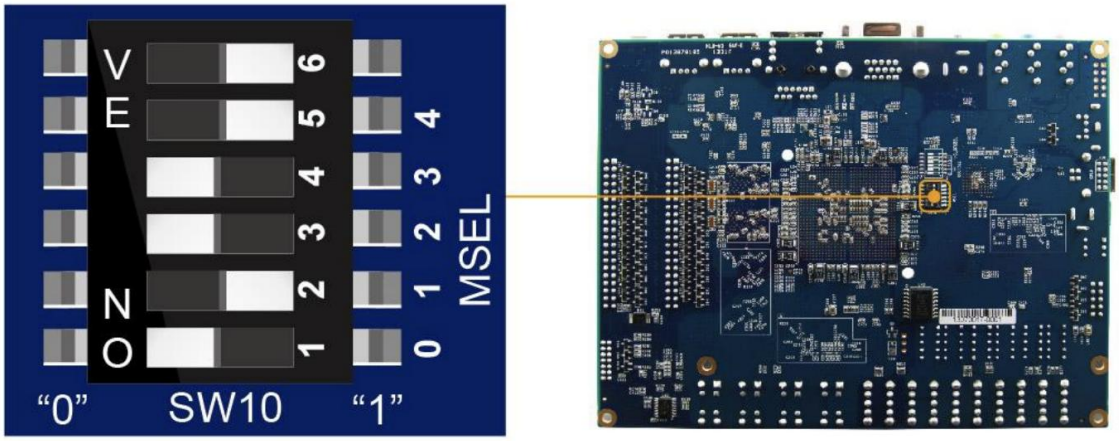


Figure E.1:SW10 configuration on the bottom side of the DE1-SoC, sourced from DE1-SoC OpenCL manual.

Insert the microSD card and power up the board. PuTTY should display Linux boot information, followed by a login prompt as shown in the Fig. E.2 below.

```
socfpga login: root
root@socfpga:~# source ./init_opencl.sh
root@socfpga:~# chmod +x boardtest_host
root@socfpga:~# aocl program /dev/acl0 boardtest.aocx
aocl program: Running reprogram from /home/root/opencl_arm32_rte/board/c5soc/arm
32/bin
Reprogramming was successful!
root@socfpga:~#
```

Figure E.2: Screenshot of Putty showing login, sourced from DE1-SoC OpenCL manual.

Next, board initialisation and file transfer steps are as follows:
- Login as ‘root’, no password required.
- Initialise OpenCL environment using ‘source ./init_opencl.sh’.
- Connect an Ethernet cable between board and computer.
- Set the computer ethernet adapter to settings to IP 192.168.0.2 and subnet mask 255.255.255.0.
- On the board, configure Ethernet by typing the following in Putty: ‘ifconfig eth0 192.168.0.2 netmask 255.255.255.0 up’.
- Use WinSCP to transfer files.

With this setup, running a project could be finally done after compiling the design files and copying them to the board. For instance, to run the board test example:
- Open command line, then navigate to:
‘cd C:\intelFPGA\18.1\hld\board\terasic\de1_soc\tests\boardtest’
- Compile the kernel (This outputs .aocx file and reports in bin/):
‘aoc device/boardtest.cl -o bin/boardtest.aocx -report -no-interleaving=default’
- To compile the host, run 'Embedded_Command_Shell.bat' from:
C:\intelFPGA\18.1\embedded
- From the launched command shell, navigate to project directory using:
‘Cd /cygdrive/C/intelFPGA/18.1/hld/board/terasic/de1_soc/tests/boardtest’
- Type 'make' to run the makefile.
- Using WinSCP, transfer the generated ‘.aocx’ and host executable to the DE1-SoC.

To run the compiled files on the board, use Putty to:
- Navigate to the project folder.
- Program the FPGA using ‘aocl program /dev/acl0 <bitstream_name>.aocx’.
- Give the host privileges to run using: ‘chmod +x ./host’.
- Finally, run the host using: ‘./host’

To use the profiler GUI, first compile the kernel with additional ‘-profile’ flag. After executing it on the board, retrieve the generated ‘profile.mon’ file from the board to the computer under the project's bin directory using WinSCP. Next, run a command shell and navigate to the bin folder and run in the command shell ‘aocl report <bitsteam_name>.aocx profile.mon’.